The Journal of Systems and Software 222 (2025) 112280

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

A structural taxonomy for lifted software product line analyses ☆

Logan Murphy *, Mahmood Saifi, Alessio Di Sandro, Marsha Chechik

University of Toronto, Toronto, Ontario, Canada

ARTICLE INFO

Keywords:
Software product lines
Lifting
Variability-aware analysis
SLR

ABSTRACT

A software product line (SPL) is a structured collection of distinct software products developed from a common set of artifacts. SPLs can encompass millions of products, so analysing each product in a brute-force manner is infeasible. To analyse SPLs directly, analyses must be *lifted*, i.e., redefined to accommodate the semantics of SPLs. Over the past two decades, many kinds of analyses have been lifted from products to SPLs. Looking at the landscape of lifted analyses, we observe various *techniques* for lifting which vary across numerous dimensions. To help engineers and research navigate this landscape, we propose a classification scheme for lifted analyses based on a set of features lifted analyses can exhibit. We then conduct a systematic literature review (SLR) analysing the landscape of lifted analyses produced over the last 20 years. We analyse 140 research papers which discuss the design and implementation of lifted analyses. We provide quantitative analysis of the types of analyses which have been lifted, and apply our taxonomy to clarify *how* lifting was accomplished. We discuss examples of how each of the lifting methods have been applied, and identify gaps in the research literature which may provide directions for future work.

1. Introduction

A *software product line* (SPL) is a family of software products derived from a common set of assets in a structured manner. Software analyses are automated procedures which produce useful information about software products. Examples of analyses include type-checking (Pierce, 2002), program slicing (Weiser, 1984), dataflow analysis (Reps et al., 1995) and model-checking (Baier and Katoen, 2008). Software analysis is a crucial task in software engineering, especially in safety-critical contexts. Unfortunately, analysing SPLs using traditional software analyses is not feasible. Traditional analyses are defined only for individual products (for instance, an individual C program), and cannot interpret the variability-aware languages in which SPLs are defined. Thus traditional software analysis must be applied in a brute-force fashion, i.e., by extracting every valid product from the SPL and analysing it individually. This is infeasible, since SPLs can encompass very large numbers (e.g., millions (Classen et al., 2011)) of distinct products. It is also wasteful, since products in the SPL will typically share significant assets with one another, resulting in redundant analysis work.

An alternative approach is to *lift* analyses from the product to the product-line level. In this context, lifting means re-implementing the analysis such that it can analyse the SPL *directly*. In particular, this re-implementation must accommodate the variability information encoded in the SPL. Over the past 20 years, a wide range of software analyses have been lifted, such as type-checking (Kästner et al., 2012a), dataflow analysis (Brabrand et al., 2012), model checking (Classen et al., 2012), change impact analysis (Shahin et al., 2021) and Datalog inference (Shahin et al., 2019), offering significant improvements in efficiency compared to the brute-force approaches.

Lifting an analysis can be a daunting task, especially given the complexity of many modern well-engineered software analysis tools. Lifting an analysis can require a significant amount of implementation effort. In safety-critical contexts, the verification of lifted analyses is also of paramount importance, which brings even further costs. Engineers who wish to lift their analyses are faced with a wide and diverse literature across many domains, making it unclear which implementation strategies for lifting are available to them, and what their tradeoffs may be. In 2014, Thüm et al. (2014) proposed a classification of variability-aware analyses for SPLs which distinguished different levels of abstraction at which the variability of an SPL is interpreted (product-based, feature-based, or family-based). While this is a central design choice when implementing variability-aware analyses, it is not the only factor an engineer may wish to consider. In particular, an engineer may which to *reuse* certain components of their existing product-level analyses as part of the lifted analysis, especially those in which significant implementation and verification efforts have already been invested. To better help engineers decide how they should implement their own lifted analyses,

☆ Editor: Laurence Duchien.

* Corresponding author.
E-mail address: lmurphy@cs.toronto.edu (L. Murphy).

https://doi.org/10.1016/j.jss.2024.112280
Received 28 March 2024; Received in revised form 11 September 2024; Accepted 9 November 2024
Available online 9 December 2024
0164-1212/© 2024 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

a classification scheme is needed which considers multiple dimensions of how analyses are designed and implemented, in addition to how variability is handled.

In a recent work (Murphy et al., 2023), we proposed a structural taxonomy of lifting methods, framed around the reuse of components of non-lifted analyses. Using a generic, formal notion of analyses and lifting, we identified four high-level implementation patterns which may induce different tradeoffs in the implementation and verification costs of the lifting. As a partial validation of our taxonomy, we conducted a preliminary survey of 38 lifted analyses and classified them using our proposed scheme. While this taxonomy could be used to distinguish alternative lifting techniques (in a manner orthogonal to Thüm et al. (2014)), it also has several limitations. First, the taxonomy only distinguishes between two variation points in the implementation of a lift: (i) whether it is white-box or grey-box, and (ii) whether the primary target of re-implementation is the specification or the interpretation of the analysis. There are yet further design choices that can and should be considered as well; for instance, some "lifted" analyses may not be maximally *precise*, in the sense of being functionally equivalent to brute-force analysis, but still be *sound* relative to the semantics of the original analysis. Second, the taxonomy assumes that every analysis can be decoupled into a specification of the analysis, and an interpretation engine. While this structure is useful for certain classes of analyses, for others it is a perspective which is neither necessary nor helpful to impose. This is a symptom of a more general limitation of the taxonomy, which is that it does not clearly define *relations* between the different design choices. Specifically, it does not clearly specify which design choices are independent of one another, which may be mutually exclusive, and which may be determined by the nature of the analysis itself.

In this work, we present a significant extension of our previous classification scheme for lifted analyses, incorporating additional design choices for lifted analyses not considered in previous schemes. More importantly, our revised scheme explicitly distinguishes between dependencies and relations between design choices and constraints faced by engineers. In the spirit of SPLE, we achieve this via feature modelling — that is, we interpret these design choices as *features*, and define a *feature model* describing the relationships between these features. This way, our classification scheme can be interpreted as a *product line of lifting techniques*. This provides a more systematic foundation for our taxonomy, and enables us to more easily identify and compare different forms of lifting, even as the number of design choices increases. While our initial taxonomy was validated via a small survey of 38 papers, we validated our revised taxonomy via a systematic literature review (SLR) of lifted analyses for software product lines. We provide a publicly available catalogue of 140 papers describing a wide range of lifted analyses. We use our taxonomy of lifting techniques to analyse the lifts in the catalogue, discuss how the different kinds of techniques have been used in different contexts, and identify gaps in the literature. To the best of our knowledge, this is the first SLR in 10 years which studies SPL analyses in a general sense (as opposed to SLRs dedicated to particular classes of analyses, such as testing (Kumar et al., 2016)).

This article is structured as follows. In Section 2, we recall basic concepts in SPLE and define an abstract, generic notion of lifting for software analyses. In Section 3, we present our novel classification scheme for lifting analyses as a *product line* of 54 lifting techniques, built from a set of 16 features organized in a hierarchical feature model. In Section 4, we define the research questions which motivate our SLR, and describe our SLR methodology. In Section 5, we report our analysis of the data collected from the SLR, providing evidence for the completeness and usefulness of our taxonomy. The results of our SLR are made public for the benefit of the SPLE community.[1]artifact We also provide some general trends and observations about the state of lifted analyses for the benefit of the broader SPLE community. We conclude with a discussion of our contributions and an outlook for future research on lifting (Section 6), and discuss related work (Section 7).

[1] https://zenodo.org/records/13744390

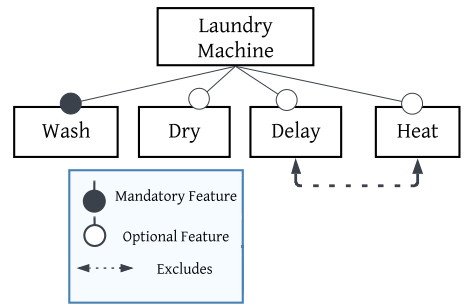

Fig. 1. A feature diagram for a product line of laundry machines.
Source: Adapted from Di Sandro et al. (2023).

2. Background

In this section, we provide background on SPLs (Section 2.1) and the lifting of analyses (Section 2.2).

2.1. Software product lines

A software product line (SPL) is a family of software products derived from a common set of assets in a structured manner. To design and implement an SPL, one requires techniques for *variability modelling* (through which variations between products are defined) and *product derivation* (through which individual products can be extracted from the SPL). Various distinct paradigms, each with their own approaches to variability modelling and product derivation, have been adopted in research and industry. In our systematic review, we consider the three most common approaches: *annotative* product lines, *delta-based* product lines, and *compositional* product lines. We begin by summarizing their common concepts.

Each approach has a foundational notion of a *feature*, meaning a unit of functionality or behaviour which may be present or absent from any particular product; as such, features are generally modelled as Boolean variables. Product line modelling usually specifies a (generally finite) set of features F. A *configuration* of features is a subset $\mathfrak{c} \in F$. Each configuration determines a particular product in the product line, namely, the one containing exactly the features in $\mathfrak{c}$. This means that, in general, a software product line can encompass up to $2^{|F|}$ distinct products.

However, not all combinations of features may actually be desirable; for instance, a designer may wish for two features to be mutually exclusive. This leads to the introduction of a *feature model*, which defines *valid* combinations of features. Feature models are usually expressed either as *feature expressions* or as *feature diagrams*.

A feature expression is a sentence in propositional logic whose atomic propositions are features in F. Each configuration $\mathfrak{c}$ determines a Boolean valuation on F, defined as $\mathfrak{c}(f) = \top$ iff $f \in \mathfrak{c}$. We use $\mathtt{Prop}(F)$ to denote the set of feature expressions over F. The satisfaction relation between a configuration $\mathfrak{c}$ and feature expression ϕ, denoted by $\mathfrak{c} \vDash \phi$, is thus defined in the standard manner. Given a feature expression ϕ, we use $[\![\phi]\!]$ to denote the set of configurations of F which satisfy ϕ. A feature model defined by feature expression Φ defines the set of configurations which are considered *valid* to be exactly those configurations which satisfy Φ. We use the notation $\mathtt{Confs}(\Phi) = \{\mathfrak{c} \in 2^F \mid \mathfrak{c} \vDash \Phi\}$ to denote this set.

A *feature diagram* is a graphical representation of the features of an SPL and their relations to one another. Feature diagrams are a widely

used notation for describing the variability of an SPL in terms of mandatory and optional features, hierarchical features (e.g., sub-features), mutually exclusive features, or dependent features.

Example 2.1. Consider a product line of laundry machines (adapted from Di Sandro et al. (2023)) with the following features: washing, drying, heating, and time-delay. We then define the set of features $F = \{\texttt{Wash}, \texttt{Dry}, \texttt{Heat}, \texttt{Delay}\}$. Suppose that we wish for the washing feature to be mandatory in all products, the drying feature to be optional, and for the heating and delay features to be mutually exclusive. We can then define our feature model $\Phi = \texttt{Wash} \wedge \neg(\texttt{Heat} \wedge \texttt{Delay})$. Consider the configuration $\mathfrak{c} = \{\texttt{Wash}, \texttt{Delay}\}$. When evaluating Φ under $\mathfrak{c}$, we set $\texttt{Wash} = \texttt{Delay} = \top$ and all other features to be $\bot$. The formula becomes $\top \wedge \neg(\bot \wedge \top) \equiv \top \wedge \neg\bot \equiv \top$, so this product is valid. In total, we have 6 valid products in this SPL. A feature diagram representing the same feature model is shown in Fig. 1.

Given a set of features and a feature model defining the set of valid configurations, a variability paradigm must provide a mechanism for defining variations and commonalities across the product space, and a mechanism for deriving products from the SPL. We summarize how these are implemented across annotative, delta-oriented, and compositional product lines.

Annotative product lines. An annotative product line begins with a set D of domain elements, such as statements in a Java program, or states in a state machine. One then defines an *annotation* function which maps each $d \in D$ to a feature expression, called the *presence condition* of d. The presence condition of an element d defines the set of configurations in which d is present. The presence condition $\top$ indicates that the element is present in all products; the presence condition $A \wedge \neg B$ indicates that the element is present only in those products equipped with feature A but not feature B, and so on. Given a configuration of features $\mathfrak{c}$, the product determined by $\mathfrak{c}$ is derived from D by removing all elements whose presence conditions are *not* satisfied by $\mathfrak{c}$. The precise mechanism for the filtering depends on the domain language of the SPL. For instance, in annotative code-based SPLs, `#ifdef` directives can be used to annotate statements with presence conditions, such that the derivation process can be performed by the compiler pre-processor. The underlying set D itself, devoid of variability annotations, is referred to as the *150% representation* of the SPL, as it typically contains strictly more elements than will be present in any one product of the SPL. Depending on the domain language, the 150% representation may or may not be a syntactically correct software artifact in its own right.

Example 2.2 (*Annotated Program*)**.** Consider the annotated C program for the laundry machine product line shown in Listing 1. Here, each program statement is annotated by one or more feature expressions. Each of the features is implemented at the macro level. When the program is compiled under a particular choice of features, the preprocessor will remove lines of code whose conditional checks fail. For instance, line 3 will be included only in products for which feature `HEAT` is present, while line 8 will be included only in products with both features `DELAY` and `DRY`. The 150% representation of this program is simply the program with all preprocessor directives removed, leaving only the C statements themselves.

Listing 1 A simple annotated program for a product line of laundry machines.

```
1  void main() {
2  #ifdef HEAT
3     warmup();
4  #endif
5  #ifdef DELAY
6      int wash_delay_t = get_delay();
7  #ifdef  DRY
8      int dry_delay_t = get_delay();
9  #endif
10 #endif
11
12 #ifdef !DELAY
13     start_wash();
14 #else
15     delayed_wash(wash_delay_t);
16 #endif
17 #ifdef DRY && !DELAY
18     dry();
19 #elif DRY && DELAY
20     delayed_dry(dry_delay_t);
21 #endif
22 }
```

Compositional product lines. While annotative product lines can be viewed as a top-down construction which filters elements out of the 150% model, *compositional* product lines use a bottom-up construction. In this paradigm, features are developed separately as *modules*, which can them be automatically composed using predetermined rules. Code-based compositional product lines are typically developed within the programming paradigms of *feature-oriented programming* (FOP) (Prehofer, 1997), and *aspect-oriented programming* (Kiczales et al., 1997).

Delta-oriented product lines. Like compositional product lines, the delta-oriented paradigm is based on the composition of modules (Schaefer et al., 2010). However, modules in delta-oriented product lines are more rich and flexible than feature modules. First, a self-contained base product – the *core module* – is defined, which may incorporate multiple features. One then defines a set of *delta modules* which define rules for modifying the behaviour of the core product. Delta models also have restrictions on the circumstances in which they can be applied.

2.2. Lifting analyses for SPLs

Lifting refers to the redefinition of the analysis such that it is able to handle the variability semantics of the SPL in a structured fashion. In particular, the analysis must be redefined such that executing the lifted analysis is functionally equivalent to performing a brute-force analysis of the SPL. We refer to this as the *correctness criterion* for lifting. Lifting analyses from the product to the product-line-level is a crucial and active area of research in SPLE, with a wide range of analyses having been lifted in recent years.

We view (product-level) software analyses as functions which take representations of software artifacts as input. "Representations of software artifacts" are very general: the inputs to analyses can be programs, models, sets of facts about programs, specifications, etc. In this article, we also take the umbrella term "analysis" to include *transformations*, e.g., transformations of software models (Strüber et al., 2017). We refer to the input space of an analysis as its *domain*, and its output space as its *codomain*. While many analyses can be modelled simply as ordinary functions on software artifacts, many others require some additional parameter or specification aimed to determine the semantics of the analysis.

Example 2.3 (*Model Checking*)**.** Model checking can be viewed as an analysis which is parameterized by a language of behavioural properties, such as Linear Temporal Logic (LTL, (Baier and Katoen, 2008)). The domain of model checking is a set of behavioural models, e.g., the set of all labelled transition systems (LTSs). The codomain of a model-checker can be modelled as a disjoint union of either a successful verification `ok`, or a counterexample trace.

Example 2.4 (*Test Case Generation*)**.** Test suite generation is a key task in software testing (Bürdek et al., 2015). A test generation analysis is specified by a coverage criterion C, which could be, for instance, a set of program locations which must be reached by the program on at least one test. Given such a criterion C, the test generation analysis takes a program P as input and returns a test suite which achieves the criterion C.

We now define a general notion of lifting analyses for SPLs. We begin with the notion of a *variational type*, which can be viewed as an algebraic generalization of the concept of an SPL. This allows instantiating our definitions across all of the aforementioned variability paradigms and all product line languages. Throughout our definitions, we implicitly fix an arbitrary set of features F and a feature model Φ.

Definition 2.1 (*Variational Type (Murphy et al., 2023)*)**.** Let X be a type. Then the type $\mathcal{X}$ is a *variational extension* of X if it is equipped with a *derivation* operator $\texttt{derive} : \mathcal{X} \to \mathsf{Confs}(\Phi) \to X$. That is, given any element of $\mathcal{X}$ and any valid configuration $\mathfrak{c}$ of features in F, a single element of X can be determined. We use $x|_{\mathfrak{c}}$ as a shorthand for $\texttt{derive}(x, \mathfrak{c})$. A type $\mathcal{X}$ which is a variational extension of some type X, is called a *variational type*.

For a given type X, there may be many reasonable variational extensions which can be implemented.

Example 2.5 (*Derivation from Annotated Programs*)**.** Consider the type $\texttt{CProg}$ of C programs, and let $\texttt{CProg}^{\#}$ denote the set of $\texttt{C}$ programs annotated with feature expressions via $\#ifdef$ directives. For instance, the program in Listing 1 represents an element of $\texttt{CProg}^{\#}$. As described earlier, given a configuration $\mathfrak{c}$, the derivation operator for $\texttt{CProg}^{\#}$ simply filters out all program statements whose annotations are not satisfied by $\mathfrak{c}$. For illustration, Listing 2 shows the product program derived from Listing 1 under configuration $\mathfrak{c} = \{\texttt{WASH}, \texttt{DELAY}, \texttt{DRY}\}$.

Listing 2 C program derived from the annotated program in Listing 1 under the configuration $\mathfrak{c} = \{\texttt{WASH}, \texttt{DRY}, \texttt{DELAY}\}$.

```
1  void main(){
2      int wash_delay_t = get_delay();
3      int dry_delay_t = get_delay();
4      delayed_wash(wash_delay_t);
5      delayed_dry(dry_delay_t);
6  }
```

Example 2.6 (*Featured Transition Systems (Beek et al., 2019)*)**.** Recall that a labelled transition system (LTS) is typically defined as a tuple $(S, \mathsf{AP}, s_0, \Delta)$, where S is a set of states, AP is a set of labels, $s_0 \in S$ is an initial state, and $\Delta \subseteq S \times \mathsf{AP} \times S$ is a transition relation (Baier and Katoen, 2008). A *Featured Transition System* (FTS) (Beek et al., 2019), intuitively, is an LTS whose transitions are annotated with presence conditions. Having fixed a feature set F and feature model Φ, an FTS is defined just as an LTS, except that the transition relation is given as $\Delta \subseteq S \times \mathsf{AP} \times \mathsf{Prop}(F) \times S$, i.e., each transition is annotated by a feature expression. Given an FTS M and configuration $\mathfrak{c} \in \mathsf{Confs}(\Phi)$, the product LTS induced by $\mathfrak{c}$ is defined as $M|_{\mathfrak{c}} = (S', \mathsf{AP}, s_0, \Delta')$, where $\Delta' = \{(s, \alpha, \phi, s') \mid (\mathfrak{c} \vDash \phi)\}$, and S' is the result of removing from S any states which are unreachable under Δ'. The 150% representation of an FTS is simply the LTS obtained by removing all feature annotations from its transitions.

Example 2.7 (*Annotative Extension (Shahin and Chechik, 2020)*)**.** Let X be some type of values. The *annotative extension*[2] of X is the type of finite sets of the form

$$\{(x_1, \phi_1), \dots (x_n, \phi_n)\}$$

where $x_i \in X$ and each ϕ_i is a presence condition such that

$$\bigwedge_{1 \le i,j \le n,\, i \ne j} [\![\phi_i]\!] \cap [\![\phi_j]\!] = \emptyset$$

and

$$\bigcup_{1 \le i \le n} [\![\phi_i]\!] = \mathsf{Confs}(\Phi)$$

The presence conditions represent disjoint sets of configurations, and every configuration which is valid under feature model Φ is represented by some presence condition. In other words, the presence conditions exactly partition the configuration space of the SPL. Since this is a generic variational type constructor, we use the notation $X^{\uparrow}$ to denote the annotative extension of any type X.

Given $x \in X^{\uparrow}$ and configuration $\mathfrak{c}$, there is a unique $(x_i, \phi_i) \in x$ such that $\mathfrak{c} \vDash \phi_i$. Thus the derivation operation from $X^{\uparrow}$ to X simply requires finding that x_i. For example, let us define $F = \{A, B\}$ and $\Phi = \top$, such that all 4 combinations of features A and B are valid. Then an element of the annotative extension of the integers could be specified as

$$k = \{(1, A \vee B), (3, \neg A \wedge \neg B)\}$$

In this case, we have $k|_{\emptyset} = 3$, and $k|_{\{A\}} = k|_{\{B\}} = k|_{\{A,B\}} = 1$.

While the name "derivation" and the preceding examples are inspired by the annotative product line paradigm, the idea can be mapped onto the compositional and delta-based paradigms. Variational types correspond to sets of features or delta modules, and the derivation operator is given by the composition rules for these modules. We are motivating our framework primarily using annotative concepts since the annotative paradigm is the most widely used in the context of lifting.[3] Having defined a generic notion of variational types, we can now formalize a generic notion of lifted analyses. In general, the lifting of an analysis from X to Y results in a function from $\mathcal{X}$ to $\mathcal{Y}$, where $\mathcal{X}$ (resp. $\mathcal{Y}$) is a variational extension of X (resp. Y). The correctness criterion for lifting, which intuitively states that the lifted analysis should be functionally equivalent to a brute-force analysis, can be formalized by asserting that the application of the analysis commutes with the derivation operator. This criterion is illustrated in Fig. 2.

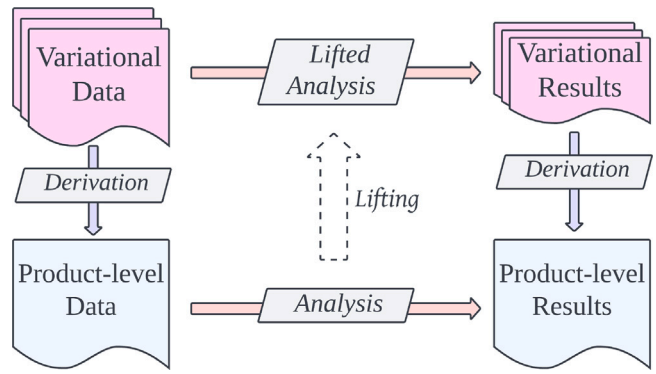

Fig. 2. An idealized illustration of lifting.

Definition 2.2 (*Lifted Analyses*)**.** Let X and Y be types, and let $\mathcal{X}$ (resp. $\mathcal{Y}$) be a variational extension of X (resp. Y). For any analysis $f : X \to Y$, a *lift* of f is a function $F : \mathcal{X} \to \mathcal{Y}$ such that

$$\forall x \in \mathcal{X}.\, \forall \mathfrak{c} \in \mathsf{Confs}(\Phi).\; F(x)|_{\mathfrak{c}} = f(x|_{\mathfrak{c}})$$

[2] This was originally called a *variability-aware type* constructor by Shahin Shahin and Chechik (2020). However, in this article, we take a much more general meaning of variability-aware (i.e., variational) types.

[3] See Section 5.

If f is an analysis engine defined using a specification language $\mathcal{L}$, i.e., f is of the form $f : X \times \mathcal{L} \to Y$, then a lift of f is a function $F : \mathcal{X} \times \mathcal{L} \to \mathcal{Y}$ such that

$$\forall x \in \mathcal{X}. \forall \mathfrak{c} \in \mathsf{Confs}(\Phi). F(x, S)|_{\mathfrak{c}} = f(x|_{\mathfrak{c}}, S)$$

In the case of analyses defined using some specification language $\mathcal{L}$, our default requirement is that the lifted analysis should be defined over the same specification language. However, some lifted analyses may also employ "variational specifications", for instance, by extending a temporal logic with modal operators encoding particular configurations (Classen et al., 2012), or by using different specifications for different configurations of products. It is relatively straightforward to modify the above definition to account for such cases.

3. A product line of lifting techniques

In this section, we propose a scheme for classifying lifted analyses based on several orthogonal aspects of the design and implementation. We begin with an overview of each of these dimensions (Section 3.1). We then organize these dimensions as *features*, and develop a structured *feature model* forming an initial universe of possible techniques for implementing lifted analyses.

3.1. Dimensions of lifted analyses

We have identified five distinct criteria against which lifted analyses can be classified and compared. The first two criteria (Section 3.1.1, Section 3.1.2) are well-established concepts in SPLE, while the other three (Section 3.1.3, Section 3.1.4, Section 3.1.5) form our part of the contribution to this space.[4] We now discuss each of these dimensions in turn.

3.1.1. Variability handling

Every lifted analysis must interpret the variability of its input in some manner. Thüm et al. (2014) distinguished three levels of abstraction to handle variability in SPL analysis: the *product*-level, the *feature*-level, and the *family*-level. We give a brief overview of these alternatives here, and refer the reader to Thüm et al. (2014) for a more extensive discussion. A *product-based* analysis decomposes the analysis of the SPL into an analysis of individual products, i.e., representations of software *without* variability. To recover SPL-level results, the analysis can either (i) analyse *all* products in a brute-forced fashion, or (ii) attempt to *generalize* the results of these product-level analyses to sets of products. These two strategies are referred to by Thüm et al. as *unoptimized* and *optimized* product-based analyses, respectively. A *feature-based* analysis attempts to analyse individual *feature modules*, i.e., domain artifacts associated with individual features, and then (if possible) compose these analysis results to form SPL-level results. Finally, a *family-based* analyses perform analysis at the level of product families (i.e., sets of products) as induced by the variation points in the product line itself. In so doing, family-based analyses attempt to reuse analysis effort across all products within each particular family. These three strategies can be further composed to create finer-grained strategies; for instance, a *feature-family-based* analysis may begin by analysing feature modules and then resort to family-based analysis to compose these results to the SPL-level. In our discussion of lifting designs, we only consider the "top-level" form of variability handling, so such an analysis would be classified as (primarily) feature-based.

Thüm et al. described several advantages and disadvantages associated with each strategy (Thüm et al., 2014). For instance, product-based analyses are easy to implement using existing tools, but generally do not scale to large SPLs; feature-based analyses are more scalable, but it may not always be feasible to compose analysis results of feature modules, resulting in incomplete analysis results; and family-based analyses are good at avoiding redundant analysis effort between related products, but require a significant up-front implementation effort.

3.1.2. Transparency of the lift

A central decision to be made when lifting an analysis is the extent to which the actual analysis semantics itself will be modified. We refer to this as the *transparency* of the lift, and distinguish between two alternatives: *white-box* lifting, in which the analysis is given completely a new semantics, and *black-box* lifting, in which the original analysis is preserved as-is, and the lifting consists of adding a *wrapper* around the analysis to handle the variability of the SPL. For this reason, we use the term "wrapping" as synonymous with black-box lifting. In truth, the phrase "black-box" is a bit misleading, since some understanding of the analysis semantics is still required to implement a wrapper.

An early discussion of the distinction between white-box and black-box lifting was given by Kästner et al. (2012b). In white-box lifting, the data structures used throughout the analyses are replaced with their *variability-aware* counter-parts, e.g., by annotating data elements with presence conditions. The internal procedures of the analysis are revised accordingly to properly interpret this variational data with respect to the analysis' intended semantics. In this work, we use a more general interpretation of "white-box": we consider any analysis which does not retain the original product-based one as a sub-procedure to be a white-box lift.

In black-box lifting, the original analysis is preserved and used as part of the lifted analysis. Since the analysis itself is not variability-aware, it must be somehow *wrapped* with some additional instrumentation for decomposing and interpreting variational data. We will consider different forms that these wrappers may take shortly; for now, we discuss reasons why an engineer may opt for either a white-box and a black-box lift. The advantages and disadvantages are very similar to those between family-based and product-based analyses — although not all black-box lifts are product-based, nor vice versa. More concretely, the appeal of white-box lifting, as argued by Kästner et al. (2012b), is that the analysis is be better able to exploit commonalities across the product space by explicitly handling variability, whereas in a black-box lift, the original analysis may pose a bottleneck as all variability is handled *outside* of the analysis. On the other hand, white-box lifts (like family-based ones) can have significant up-front costs, whereas black-box lifts (by definition) leverage existing analysis artifacts, with the engineer only being responsible for implementing the wrapper. This may be particularly attractive if the existing analysis is equipped with sophisticated optimizations, or if it is an off-the-shelf artifact, and the engineer wishing to lift it does not have the requisite expertise to perform a white-box lift.

Having distinguished white-box from black-box lifts and considered their potential usage trade-offs, we now discuss in more detail how black-box lifts are actually implemented.

3.1.3. Structuring black-box lifts

Suppose that an engineer has decided that she wants to pursue a black-box lifting strategy. As we have described above, she is responsible for implementing a *wrapper*. We distinguish between two alternative types of wrappers she may wish to pursue, based on the number of times the original product-level analysis is invoked. It can be invoked only once, in which case we are considering an *all-at-once* wrapping strategy, or (potentially) multiple times, resulting in an *incremental* wrapping strategy.

[4] A version of the material covered in Section 3.1.4 was presented in our preceding work (Murphy et al., 2023).

All-at-once wrappers. As the name suggests, an all-at-once wrapping strategy attempts to analyse the entire product line during a single pass of the original product-level analysis. This is often achieved through the use of *variability encoding*, which is an embedding of the SPL's variability (e.g., as defined by presence conditions) into a product-level representation which can be digested by the analysis.

For instance, given SPLs defined as annotated source programs, one can perform code-level rewriting such that SPL features become Boolean program variables and preprocessor directives (e.g., `#ifdef`) become control flow statements (e.g., conditionals) (Post and Sinz, 2008a). A similar procedure can be done for symbolic model checking, as presence conditions on transitions can be encoded as propositional guards in the specification of the model (Classen et al., 2011). The result of the variability encoding is an object which, in some form, reflects the possible behaviours of all products (this is in contrast with the 150% representation which can exhibit behaviours not observed in *any* product). Such an object is referred to in the literature as a *product simulator* (Castro et al., 2021), a *variant simulator* (von Rhein et al., 2016), or a *metaproduct* (Meinicke et al., 2014). In this work, we use the term *metaproduct*.

Once a metaproduct has been generated, the original analysis will necessarily return a (product-level) result. Depending on the needs of the engineer, this may be sufficient — for instance, if the analysis is a form of verification, and the variability encoding has been implemented properly, a successful verification of the metaproduct can ensure that every product of the SPL is effectively verified. But in general, the engineer may wish to extract separate analysis results for different sets of products in the SPL, which generally requires multiple *incremental* executions of the analysis.

Incremental wrappers. While all-at-once wrappers attempt to translate the SPL into a metaproduct which is analysed once, an *incremental* wrapper either analyzes the metaproduct multiple times, or otherwise analyzes a set of products (or metaproducts) one by one. Compared to the all-at-once wrapping, this makes it much more straightforward to produce a variability-aware analysis result for the SPL by aggregating the results of each analysis instance until the entire product space has been covered. We emphasize that incremental wrapping is *not* necessarily synonymous with product-based analysis, since the wrapper may achieve family-level reasoning through the use of variability encoding.

Example 3.1 (*An Incremental Wrapper for Variational Satisfiability Checking*)**.** Young et al. provided a way to wrap SAT solvers to be variability-aware (Young et al., 2020). The wrapper iteratively decomposes a variability-aware propositional formula into its variant subformulae, such that each decomposition can be mapped to a family of configurations. Satisfiability is checked whenever a subformula variant is fully decomposed. The wrapper extracts a (non-lifted) model or unsatisfiability proof from the base solver for each configuration, and aggregates these models into a variational model/proof.

As suggested in Example 3.1, the structure of an SPL representation can sometimes yield very natural (if not always efficient) implementations of incremental wrappers. To further highlight this point, let us define a procedure for lifting *any* binary function $f : X \times Y \to Z$ via a incremental wrapper.

Example 3.2 (*Lifting Graph-based Model Transformations*)**.** One of the fundamental tools in model-based software engineering is *model transformation*. Salay et al. provided a incremental wrapper for the Henshin transformation engine (Salay et al., 2014). Given the transformation T and variability-aware model m, the wrapper rewrites T to a modified transformation T' which over-approximates the semantics of T for the variants in m. The transformation is applied incrementally to various parts of the model. Unlike the previous example, parts of the modified transformation which will correspond only to invalid products are filtered *before* the modified transformation is applied. The result of the partial application are then aggregated into the variability-aware target model.

Data: $(x, y) \in X^{\uparrow} \times Y^{\uparrow}$
Result: $z \in Z^{\uparrow}$
$z \leftarrow \emptyset$;
foreach $(x_i, \phi_i) \in x$ **do**
  foreach $(y_j, \psi_j) \in y$ **do**
    if $SAT(\phi_i \wedge \psi_j)$ **then**
      $z \leftarrow z \cup \{ (f(x_i, y_j), \phi_i \wedge \psi_j) \}$
    end
  end
end

Algorithm 1: A generic, incremental wrapper for lifting a binary function $f : X \times Y \to Z$.

Example 3.3 (*Generic Incremental Lift*)**.** Assume that the SPL representation of the types X and Y is via the annotative type constructor defined in Example 2.7. Our construction (Algorithm 1) is an imperative reformulation of Shahin's functional "shallow lifting" (Shahin and Chechik, 2020). The input to the wrapped function is a pair (x, y), where each of x and y is a finite set of annotated values whose annotations form a partition of the configuration space. The wrapper iterates through the cross product of the two inputs, i.e., trying every combination of values in x against values in y. For each combination of annotated values, it checks to see if the conjunction of their annotations is a satisfiable feature expression. If so, it invokes f and records the result with the conjunction as its annotation. The correctness of Algorithm 1 as a lift is straightforward: one just proves that the annotations of elements of z incrementally form a partition of the configuration space.

In this case, the explicit annotative construction is a very convenient SPL representation. However, the scalability of our construction is very closely tied to the shape of its inputs. We could consider a worst-case scenario in which there is no sharing among the inputs, i.e., there are $2^{|F|}$ elements in x and y, each of which representing exactly one configuration of features. In principle, some incremental wrappers such as this one could devolve into brute-force execution. This is in contrast with an all-at-once wrapper, for which execution is guaranteed to be happen only once.

3.1.4. Language-based lifting

So far, we have identified existing well-known distinctions between lifting strategies (variability handling and transparency), and considered two alternative strategies for implementing wrappers for black-box lifting. Some choice from each of these dimensions is mandatory for any lifted analysis. We now consider an *optional* design choice which may present itself to the engineer, depending on the nature of the analysis itself: specifically, depending on whether or not the analysis is *language-based*. Recall from Definition 2.2 that a language-based analysis takes not only the SPL as input, but also a *specification* defining the "rules" of the analysis. In this work, we take a very general view of specification languages for language-based analyses, so that temporal logic formulae (Classen et al., 2010), specifications of dataflow analyses (Bodden et al., 2013), Datalog queries (Shahin et al., 2019), OCL constraints (Czarnecki and Pietroszek, 2006) and test coverage criteria (Lity et al., 2019) are all possible examples of analysis specifications. The unifying feature is that the lifted analysis must consider both the specification of the analysis and the SPL data itself to produce a result, and that the lifted analysis must be correct for *every* analysis specifiable in the language.

When lifting a language-based analysis, the engineer has an additional potential design choice not available to other analyses: rather than modifying or decomposing the SPL to produce an analysis result, the engineer may be able to instead modify the specification of the analysis. We refer to the first strategy of lifting as *data-oriented*, as it

primarily operates on the variational data (the SPL) provided to it as input, and the second strategy as *specification-oriented*. Data-oriented lifting can be generally viewed as the "default case" of language-based lifting; like lifts for non-language-based analyses, either variational data structures (for white-box lifts) or variability encoding (for black-box lifts) can be used.

Example 3.4 (*Lifted Model Checking*)**.** Foundational work on product line model checking was done by Classen et al. (2010), in which FTSs were introduced as a language for behavioural modelling of SPLs. The authors provide a new algorithm for verification of FTSs against LTL properties, by adapting a classic LTL verification algorithm based on Büchi product automata (Baier and Katoen, 2008). This white-box approach was later developed into the well-known lifted model checker SNIP (Classen et al., 2013), which can be used to verify an FTS against any LTL specification. This reinterpretation of model checking for FTS was also later extended to real-time model checking (Cordy et al., 2012).

Example 3.5 (*Lifted Datalog Analysis*)**.** Shahin et al. (2019) implemented a reinterpretation of the Datalog-based analysis engine `Souffle`. To do this, they implemented a modified version of forward chaining to handle variability-aware facts. By lifting the Datalog engine, any Datalog analysis executed on the lifted engine is automatically made variability-aware.

However, for some language-based analyses, there is an alternative to data-oriented lifting which is quite dissimilar from other forms. In *specification-oriented* lifting, the specification of the analysis is automatically rewritten into one which can accommodate the variability in the SPL. In general, we can view this rewriting process as a mapping from a source specification language, $\mathcal{L}$, to a target language, $\mathcal{L}'$, such that $\mathcal{L}'$ can be assigned "variational semantics". Given such a rewriting system $\mathcal{R}$, we may ask how we should actually execute the rewritten analysis $\mathcal{R}(S)$, since any interpreter for the original analysis in $\mathcal{L}$ would only be able to interpret product-level data. In practice, one may want to reuse this interpreter (or only make minor modifications to it), in which case the language $\mathcal{L}'$ may be defined as a simple extension of $\mathcal{L}$, e.g., by adding some new variational type constructor to it. Generally speaking, such specification-based lifts are most suitable for specification languages which are already highly expressive (e.g., specifications of abstract interpretations or programming languages).

Example 3.6 (*Lifting Functional Programs*)**.** Shahin et al. lifted a simple (but Turing-complete) functional language, PCF+ (Shahin and Chechik, 2020), by rewriting all the syntactic program constructs (function application, pattern matching, fixpoint operator, etc.) to be variability-aware. This allows any analysis written in PCF+ to be automatically lifted. In their implementation, they used the Haskell interpreter for both the ordinary and variability-aware semantics. The authors also proved the correctness of the rewriter, so every analysis which is automatically lifted is also automatically proven correct.

Example 3.7 (*Lifting Model Queries*)**.** The Viatra Query Language (VQL) is a language for specifying pattern-based queries for software models (Varró et al., 2016). Di Sandro et al. (2023) provide a wrapper for lifting VQL queries to operate on product line models. When a product-level query is applied to a product line model, the wrapping mechanism intercepts the query specification and performs a series of rewrite tasks to account for the variability-aware metamodel to which the target model conforms. The modified query does not in itself become variability-aware, but rather aggregates all model elements together, and so provides an over-approximation of the idealized variability-aware query. The Viatra query engine is then invoked on the modified query, and its results are aggregated based on the input model's variability annotations. Results corresponding to invalid product configurations are filtered out.

3.1.5. Precision of lifted analyses

The final design choice for lifted analyses we consider was already mentioned in the discussion of black-box lifting (Section 3.1.3), which is the *precision* of the lifted analysis. By "precision", we mean the degree to which the lifted analysis satisfies the correctness specification in Definition 2.2. This specification asserts that applying the lifted analysis to an SPL must produce a variability-aware value, from which a set of product-level values can be derived, and these derived values must be *exactly* those that would have been produced by applying the product-level analysis to their analogous products in the SPL. This is a strong requirement, and some use cases for lifted analyses may be satisfied by weaker forms of lifting. More specifically, a "lifted" analysis may be sufficient if it is at least *sound* with respect to some correctness property of the original analysis. We formalize this distinction with the following alternative to Definition 2.2:

Definition 3.1 (*Weak Lift*)**.** Let $f : \mathcal{X} \to \mathcal{Y}$ be some analysis, let P be a predicate over $\mathcal{Y}$, and let Q be a predicate over $\mathcal{X}$ such that

$$\forall x \in \mathcal{X}.(P \circ f)(x) \implies Q(x).$$

Then an analysis $F : \mathcal{X}^{\uparrow} \to \mathcal{Y}$ is a *weak lift* of f iff

$$\forall x \in \mathcal{X}^{\uparrow}.\,(P \circ F)(x) \implies \forall \mathfrak{c} \in \mathsf{Confs}(\boldsymbol{\Phi}).Q(x|_{\mathfrak{c}}).$$

Intuitively, Definition 3.1 means that if running an analysis f always determines whether or not Q holds for a software product, then running weak lifted F on an SPL determines whether or not Q holds for every product in the SPL — but it is not guaranteed to tell you anything else. In particular, if Q does *not* hold for every product in the SPL, a weak lifted analysis may not indicate exactly for *which* products Q fails.

Example 3.8 (*Weak Lifted Model Checking*)**.** Fix some product based model checker V which takes as input any LTS M and LTL specification ϕ, and returns either `ok`, indicating a successful verification, our a counterexample trace revealing a violation of ϕ in M. More precisely, the key soundness specification for V is that for any specification ϕ,

$$\forall M \in \text{LTS}.\, V(x) = \texttt{ok} \implies Traces(M) \subseteq Words(\phi) \tag{1}$$

Now suppose that we use a black box lifted model checking procedure, denoted $\mathcal{V}$, in which (i) a simple variability encoding is used to translate FTSs to (symbolic) LTSs, in which features become state variables in the model and presence conditions on transitions become propositional transition guards, and (ii) we execute V on the resulting LTS. Let $\widehat{M}$ be such an FTS, and let M be its resulting LTS metaproduct. If V returns `ok`, then by the assumption of (1), we know that every configuration of $\widehat{M}$ satisfies ϕ. In other words, the following holds:

$$\forall \widehat{M} \in \text{FTS}.\, \mathcal{V}(x) = \texttt{ok} \implies$$
$$\forall \mathfrak{c} \in \mathsf{Confs}(\boldsymbol{\Phi}).Traces(\widehat{M}|_{\mathfrak{c}}) \subseteq Words(\phi) \tag{2}$$

with (2) being an instantiation of the criterion of Definition 3.1. However, this analysis is not equivalent to performing brute-forced analysis. Suppose that there are multiple distinct products in $\widehat{M}$ violating ϕ. The wrapper $\mathcal{V}$ identifies only *one* such violation, while a brute-forced analysis identifying *all* violations. For this reason, $\mathcal{V}$ qualifies only as a weak lift, and not a true one.

Generally speaking, implementing a weak might be significantly easier than a true lift, at the expense of completeness of information in the analysis result. Furthermore, some important properties of lifted analyses do not hold for weak lifted analyses. For example, in our prior work, we proved that any workflow (i.e., composition) of lifted analyses inherits all functional correctness properties from the analogous product-level workflow. However, this only holds if each of the functions in the workflow are *true lifts*, as the proof relies crucially on the compositionality of (true) lifted functions. This does not mean that weak lifted analyses cannot be incorporated as part of lifted workflows, but rather that proving the correctness of this workflow must be done on a case-by-case basis, depending on which properties are preserved by the weak lift.

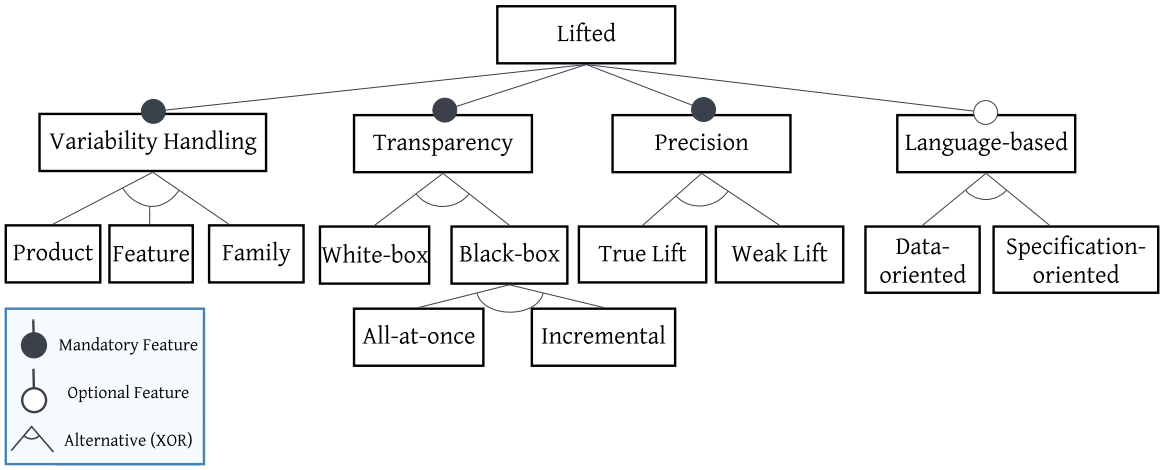

Fig. 3. A feature model describing design choices encountered when lifting analyses.

3.2. Comparing lifting alternatives

In the preceding discussion, we have mentioned potential reasons why a developer may opt to choose between one lifting alternative or another (e.g., white-box lifting vs. black-box lifting), some of which have been previously articulated by other researchers (Kästner et al., 2012b; Thüm et al., 2014). However, it is difficult to make sweeping generalizations about the relative advantages and disadvantages of lifting strategies from first principles. Instead, such claims should be backed by concrete empirical evidence, in which alternative lifting strategies are systematically compared over a single benchmark. While such studies are relatively rare, there are cases of researchers performing this kind of comparative analysis.

Variational execution (Nguyen et al., 2014) attempts to reduce the cost of actually running the tests by executing multiple programs "simultaneously". The intent is to test overlapping computations only once, reusing their results when executing distinct behaviours. Varex (Nguyen et al., 2014) is a variability-aware interpreter for PHP which forks execution upon entering annotated code blocks, and then merges the results upon completion. It can be viewed as an incremental wrapper for standard PHP execution, since multiple executions are performed for different product families. A similar interpreter for Java, called VarexJ (Meinicke, 2014), was developed as well. A significant improvement was made with VarexC (Wong et al., 2018), which performs an embedding of Java bytecode into equivalent "variational" bytecode for execution directly on the Java Virtual Machine in a single pass (i.e., an all-at-once wrapper). This latter approach provides a speedup factor of up to 46 compared to the splitting-based (i.e., modular) approach, providing some empirical evidence that all-at-once wrappers can offer better performance over incremental ones.

In another analysis domain, Cordy et al. compared two approaches to SPL synthesis, one of which being a black-box incremental lift (Greenyer et al., 2013), and the other being a white-box lift using a reimplementation of a traditional synthesis algorithm for Featured Büchi Automata (Cordy et al., 2015). The authors found that the white-box strategy induced a speedup of 1.8 compared to the incremental strategy.

3.3. A feature model for lifting techniques

Thus far, we have identified five dimensions of variation across lifted analyses, each of which can be manifested in one of several alternative choices. If we view these alternative design choices as *features* of the lifting process, we can define a "product line of lifting techniques" by organizing these features into a feature model. To define a feature model, one must determine not only which features are mandatory or optional, but which features may be dependent or exclusive of one another. To capture the widest possible range of potential lifting techniques, our initial feature model assumes only the following constraints:

- Each of the alternatives within the five dimensions defined in Section 3.1 are *logical* alternatives (e.g., a lift must be either white-box or black-box, but not both).
- Each of the dimensions induces a mandatory choice of alternatives, *except* for the options for language-based lifting (data-oriented vs. specification-oriented). This reflects the fact that not every analysis is language-based.

The resulting feature model is shown in Fig. 3. It is very expressive, allowing for up to 54 unique configurations, each of which corresponding to a (theoretical) strategy for implementing a lift. Of course, we expect that in practice, many of these combinations are unlikely to be pursued, but this would not justify us from excluding their theoretical possibility. Instead, a systematic evaluation of how lifting is actually done in practice is needed — not only to evaluate the accuracy of our feature model, but also to determine whether this perspective on lifting can give us insights into how it is actually performed by researchers and engineers.

3.4. Summary

In this section, we have proposed a conceptual framework for distinguishing and classifying techniques for lifting product-level analyses. In particular, we defined an initial feature model defining a large set of potential techniques for implementing lifts. We emphasize that our proposal here is not meant to capture *all* relevant aspects of lifted analysis design — we are only proposing an initial framework to which additional features can be added. While it is difficult to make general assertions about the benefits of different lifting techniques, our proposal includes rationale for why engineers may opt for some features of lifted analyses over others (e.g., reuse of the implementation effort of black-box lifts).

4. A systematic literature review for lifting

In the previous section, we proposed a scheme for classifying lifted analyses by identifying various orthogonal *features* of the analysis, such as the transparency and precision of the lift. In this section, we evaluate the applicability and usefulness of our classification scheme by means of a systematic literature review (SLR). Our evaluation is framed around the five research questions (RQs) shown in Table 1. These RQs can be divided into two categories.

Table 1
Research questions aiming to (i) identify how closely our proposed classification scheme for lifting matches the research literature (**RQ1–RQ2**), and (ii) employ our classification scheme to gain new insight into how lifting is performed in practice (**RQ3–RQ5**).

RQ1.	How many of the configurations allowed by our (initial) feature model have been implemented in practice?
RQ2.	What other potential feature constraints can be gleaned from the research literature?
RQ3.	Across all lifted analyses, what lifting features and combinations thereof are employed most frequently?
RQ4.	Does the frequency of features or combinations thereof vary depending on the kind of analysis being lifted?
RQ5.	Is there any relationship between the implementation of a lift and the presence of a correctness proof?

Our first two research questions both aim to compare the *potential* methods for lifting analyses we have identified in Section 3 against the lifted analyses which have *actually* been used in practice. More specifically, both **RQ1** and **RQ2** are chosen based on the assumption that not every configuration allowed by the feature model shown in Fig. 3. **RQ1** aims to measure how much our feature model over-approximates the space of lifted analyses, while **RQ2** aims to identify additional feature constraints which can be used to refine our feature model, shrinking the gap between the theorized and real-world lifting strategies.

The remaining research questions (**RQ3**-**RQ5**) instead attempt to *leverage* our classification scheme in order to better understand how lifting is implemented by practitioners. **RQ3** aims to identify which choices of alternative features – and which specific configurations of features – are most commonly employed. **RQ4** also aims to gauge the popularity of particular lifting strategies, but at the level of specific *classes of analyses* (e.g., static code analysis, model checking). Finally, **RQ5** attempts to identify whether the choice of how lifting is implemented may impact the likelihood of the lift being accompanied by a correctness proof.

We now describe the execution of the SLR.

4.1. Survey methodology

Following Wohlin et al. (2022), we conducted a hybrid SLR consisting of four phases:

1. A *database search* phase, in which a string query is used to gather relevant papers from digital academic repositories.
2. A *manual evaluation* phase, in which candidate papers are filtered out based on inclusion/exclusion criteria, and labelled based on predefined qualitative criteria.
3. A *snowballing* phase, in which references and citations of a judiciously chosen sample of included papers are iteratively searched to find new candidates.
4. A second labelling phase, where the papers selected for inclusion from step (3) are labelled in the same manner as in step (2).

As noted by Wohlin et al. (2022), using both database searching and snowballing generally results in more papers being collected than using only one of the two collection methods. Our high-level survey methodology is shown in Fig. 4, and the number of papers added/removed from each stage is recorded in Table 2.

Inclusion criteria. In order for a paper to be included in our survey, each of the following inclusion criteria must have been met:

I1. The paper must be written in English.
I2. The paper must have appeared in peer-reviewed conference proceedings or in a peer-reviewed academic journal.
I3. The paper must have been published between January 2005 and March 2024, inclusive.

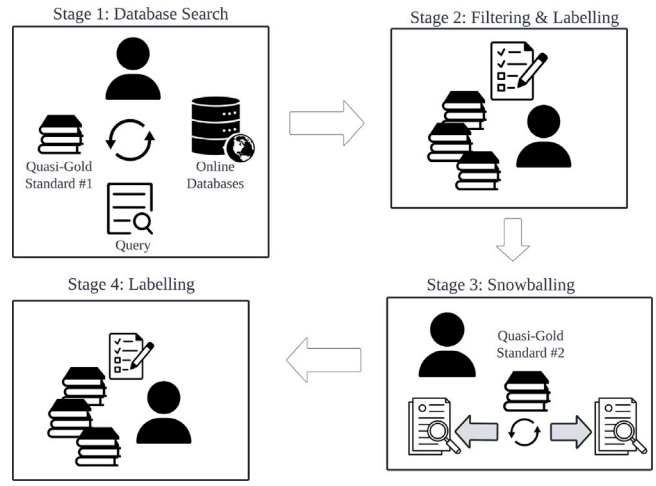

Fig. 4. Outline of our data collection process.

I4. The paper must describe the *lifting* (or weak lifting) of a product-level analysis or operation to SPLs.
I5. The analysis must be defined over *static* product lines.

Criteria **I1–I3** are standard requirements for an SLR. The starting year of 2005 was chosen to limit our survey to a 20-year interval; furthermore, from preliminary investigations which found a significant increase in research related to lifting beginning around 2010. Criterion **I4** is the most crucial point for our SLR. Not every analysis or operator defined on SPLs can be considered a lifted analysis. Many analyses and operations, such as feature model analysis (Benavides et al., 2010), can be defined *only* on SPLs, and have no product-level analog. Furthermore, for an analysis to be considered *lifted*, its semantics must correspond to either the correctness specification for true lifting (Definition 2.2) or weak lifting (Definition 3.1). Criterion **I5** implies that we omit analyses which are defined over *dynamic* product lines, i.e., those in which a product's feature configuration can change during its execution.

Additionally, a paper was excluded from our survey if it met any of the following exclusion criteria:

E1. The paper is a survey, literature review, industrial report, experience report, doctoral symposium report or tutorial paper.
E2. The lifted analysis is insufficiently formalized or relies primarily on human/manual analysis.

The exclusion criterion **E1** allowed us to focus on research papers, which is where we expect to find descriptions of novel lifted analyses. Criterion **E2** allowed us to focus only on analyses which can be automated *in principle*. Even if an analysis is not implemented in a tool, an algorithm provided in pseudocode was considered "sufficiently formal" for the sake of **E2**.

4.2. Database search

In the database search phase, we performed incremental query expansion (Badami et al., 2023) based on an initial quasi-gold standard (QSG). The initial QSG is a set of 36 papers based on a preliminary (non-systematic) survey of lifted analyses in our preceding paper (Murphy et al., 2023). Our database search was conducted with the ACM DL, Springer, IEEEXplore, and Scopus digital repositories. We used a string-based search targeting document titles and abstracts. We began with an initial search string which matched combinations of the phrases "lifting", "variability-aware" and "product lines", and combinations of any of these phrases with "analysis", "testing", or "verification". We also configured our database query searches to

Table 2
Number of papers added/removed during each stage of the SLR.

Stage	Quantity
Database Search	2327
Manual Filtering	−2238
Snowballing	+51
Final	140

incorporate our inclusion critiera **I1–I3**. We then incrementally refined our string until every entry in our QGS could be identified from the query, typically, by adding phrases for particular analysis (e.g., "synthesis") and synonyms for product-line-level analyses (e.g., family-based", "configuration-preserving"). Once our query refinement had converged, we were able to collect 2782 papers and conducted our first stage of manual evaluation. The manual evaluation consisting of a filtering phase and a labelling phase.

4.3. Manual filtering

While our database query was able to verify that the selected papers satisfied inclusion criteria **I1–I3**, criteria **I4** and **I5** needed to be ascertained manually. We first performed lightweight screening, inspecting the titles and (if necessary) the abstracts of the papers to determine whether they were potentially related to the analysis of SPLs (but not necessarily related to lifted analysis). Our database query, while comprehensive with respect to our QGS, also returned many papers which were not relevant to SPL analysis, which were easy to identify and exclude. Only a minority of the original 2327 papers (approx. 25%) were retained for closer scrutiny, i.e., a detailed reading to determine whether **I4** was satisfied, i.e., that the paper reported on a *lifted* (or weak lifted) analysis, and not on an SPL-specific analysis.

De-duplication. Many of the analyses which have been lifted by the SPL community have been discussed in a series of publications, e.g., conference papers followed by journal extensions, case studies or tool papers. During our filtering process, we kept an extended version of a paper in the inclusion set only if it used a different combination of lifting features compared to the original presentation. For "duplicates" of papers for which this was not the case, we kept only the most recent publication describing the lifted analysis.

In total, our manual filtering of the papers identified via database search resulted in 2237 papers being filtered out, leaving a net of 89 papers. We then proceeded to the labelling stage of our manual evaluation.

4.4. Data labelling

Having selected our initial set of papers for inclusion in our survey, we began manually labelling them in order to obtain sufficient data to answer our research questions. In addition to bibliographic data (e.g., year and venue of publication), we assigned labels for each of the following six classes:

C1. Variability handling
C2. Transparency of lifting
C3. Language-based analysis
C4. Precision of lifting
C5. Analysis category
C6. Correctness proof

Each of **C1–C4** reflects one of the sets of alternative lifting features discussed in Section 3. Each of these four criteria was generally straightforward to determine for each lift, with **C1** and **C2** often being noted explicitly by the authors, and **C3** often being clear from the definition of the analysis. For **C3**, we classify each analysis as either data-oriented, specification-oriented, or "N/A" if it is not a language-based. Class **C5** refers to the *type* of analysis being lifted in the paper. Here, an "analysis type" refers to a broad class of analyses which are related by their methods or use cases — see below. Finally, class **C6** refers to whether or not the lifted analysis is accompanied by a correctness proof of the lift (cf. Definition 2.2 for true lifts and Definition 3.1 for weak lifts.)

For the sake of our survey, we defined the following six categories of analyses (class **C5**):

Static analyses. A static analysis is any automated procedure which operates directly on source code and returns "facts" or "information" about the software product. For the sake of this survey, we distinguish between "lightweight" static analysis and *formal verification*, i.e., model checking or deductive verification. Examples of so-called lightweight static analyses include type-checking (Pierce, 2002), dataflow analysis (Reps et al., 1995), and program slicing (Weiser, 1984). We also consider parsing to be a form of static analysis for the sake of the survey.

Model checking. By model checking, we mean classic model checking procedures on transition systems (Baier and Katoen, 2008), as well as its variations, such as model-checking of timed automata (Yovine, 1996) and probabilistic model checking (Katoen, 2016).

Testing. Under the umbrella of testing, we consider any analysis or operation which serves to facilitate *dynamic* verification. Examples include test suite generation (Bürdek et al., 2015), and dynamic analysis (Kästner et al., 2012b; Meinicke, 2014). We recall that, as per inclusion criterion **I4**, testing techniques which are intended to be incomplete with respect to the configuration space (e.g., product sampling) were not considered "lifts".

Model-based analyses. This class encompasses all analyses, operators and transformations which operate on software models (as opposed to code), excluding those which can be considered model checking. These include the analysis of model constraints (Richters and Gogolla, 2000), model queries (Varró et al., 2016) and model transformations (Strüber et al., 2017).

Theorem proving. We use *theorem proving* as an umbrella term to encompass both automated theorem proving, e.g., SAT and SMT solving, as well as *deductive* software verification — verification of software implementations against formal specifications using a formal proof system. Deductive verification can employ theorem provers and proof assistants such as KeY (Ahrendt and Grebing, 2016), Coq (Paulin-Mohring, 2011) and Lean (Moura and Ullrich, 2021).

Synthesis. By synthesis, we mean the automatic generation of a software product (code or model) from some kind of high-level specification, such that the synthesized product conforms to that specification. The specification can, for instance, take the form of a logical formula (Manna and Waldinger, 1980) or software model (Greenyer and Kindler, 2013).

4.5. Snowballing

Having labelled our selection of included papers, we began a second phase of data collection through forward and backward snowballing. To minimize any sampling bias incurred during our database search, we selected a new quasi-gold standard (QGS) of 18 papers from our inclusion set, following the snowballing guidelines provided by Wohlin Wohlin (2014) to avoid biases in the data collection. Specifically, we made sure to select papers from different sets of authors and publication venues, and spread out over the time period considered in our survey. We also ensured a diversity of analysis classes by requiring at least two papers from each type of analysis. This second QGS-2 was used as the new starting set for forward-and-backward snowballing (Wohlin et al., 2022), i.e., for each paper in the new starting set, we looked

Table 3
Publication venues of selected papers. Only publication venues corresponding to two or more papers are listed.

Publication venue	%
Systems and Software Product Lines Conference (SPLC)	13.6%
Science of Computer Programming (SCICO)	7.9%
International Conference on Software Engineering (ICSE)	6.4%
International Conference on Automated Software Engineering (ASE)	6.4%
Generative Programming and Component Engineering (GPCE)	5.7%
International Conference on the Foundations of Software Engineering (FSE)	4.3%
International Conference on Variability Modelling of Software-Intensive Systems (VAMOS)	4.3%
International Conference on Fundamental Approaches to Software Engineering (FASE)	2.9%
Journal of Systems and Software (JSS)	2.9%
International Workshop on Feature-Oriented Software Development (FOSD)	1.4%
Transactions on Software Engineering (TSE)	1.4%
Transactions on Software Engineering and Methodology (TOSEM)	2.1%
Journal of Logical and Algebraic Methods in Programming (JLAMP)	2.1%
International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISOLA)	2.1%
International Conference on Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)	2.1%
International Journal on Software Tools for Technology Transfer (STTT)	1.4%
International Conference on Requirements Engineering (RE)	1.4%
Information and Software Technology (IST)	1.4%
International Conference on Functional Programming (ICFP)	1.4%
Conference on Programming Language Design and Implementation (PLDI)	1.4%
Empirical Software Engineering (EMSE)	1.4%
International Conference on Integrated Formal Methods (iFM)	1.4%
Formal Methods for Open Object-Based Distributed Systems (FMOODS)	1.4%
Other	22.1%

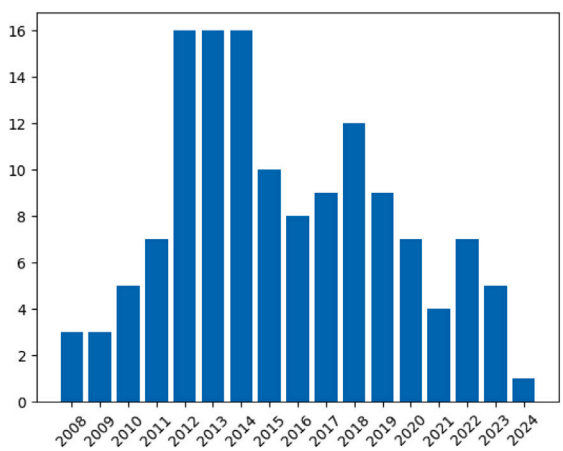

Fig. 5. Publication years of the surveyed papers.

at its references (backward snowballing) and the papers which cited it (forward snowballing). Based on paper titles, abstracts, and keywords, we identified potential new candidates, which were then subject to the evaluation criteria described above. Papers which were worthy of inclusion were used to form the start set of the subsequent iteration. We conducted three rounds of snowballing, yielding an additional 51 papers for inclusion in our survey. Once we identified papers for inclusion from snowballing, we repeated the labelling process defined in Section 4.4 for the new papers.

5. Survey results & evaluation

In this section, we report answers to the RQs based on our analysis of the papers collected through our systematic literature review. As described in Section 4, we are studying two sets of research questions. **RQ1** and **RQ2** are used to validate the completeness and usefulness of our proposed taxonomy, respectively; **RQ3–RQ5** ask more general questions about the landscape of lifted analyses, for the broader interest of the SPLE community. We begin by reporting basic bibliographic metrics of our surveyed papers.

Bibliographic metrics. Fig. 5 illustrates the distribution of papers into the years 2005–2024, which is the interval considered in our survey. As shown in the figure, this time period saw a peak in the introduction of lifted analyses in 2012–2014. Since 2021, the number of lifted analyses introduced each year has averaged at about six.[5]

Table 3 shows the publication venues (conference proceedings and journals) in which the majority of our papers were published. To keep the table concise, we only report venues which account for two or more papers in our dataset; venues which provided only a single paper are aggregated under "Other". The table reports a range of conferences and journals. The most frequently represented venues include SPL-specific conferences (e.g., SPLC, VAMOS), top-tier software engineering conferences (e.g., ICSE, ASE, FSE), and software engineering and computing journals (e.g., Science of Computer Programming, JSS).

5.1. RQ1: The precision of the initial feature model

Recall from Section 3.3 that our initial (permissive) feature model for lifting techniques encompasses 54 valid configurations. Reviewing our labelled repository of lifted analyses, we found a total of 24 distinct combinations of the features, yielding roughly a 44% coverage of the configuration space. Thus, the naive feature model shown in Fig. 3 significantly overestimates which combinations of features are likely to be chosen by practitioners, as most of the configurations allowed by this feature are not represented by any paper in our SLR.

5.2. RQ2: Identifying practical feature model constraints

By inspecting the set of configurations which are actually observed in the literature, we can identify additional cross-cutting feature constraints not defined in our initial feature model. In total, we identified 11 additional feature constraints which together reduce the set of valid configurations from the initial feature model to exactly match those observed in the survey. However, many of these constraints involve 3–5 features, making it difficult to deduce in general terms why these constraints may exist. Here, we report three *binary* feature constraints, which we believe can be easier to explain *post hoc*.

[5] The count for 2024 is an outlier, since this article is only up-to-date with papers published by March 2024

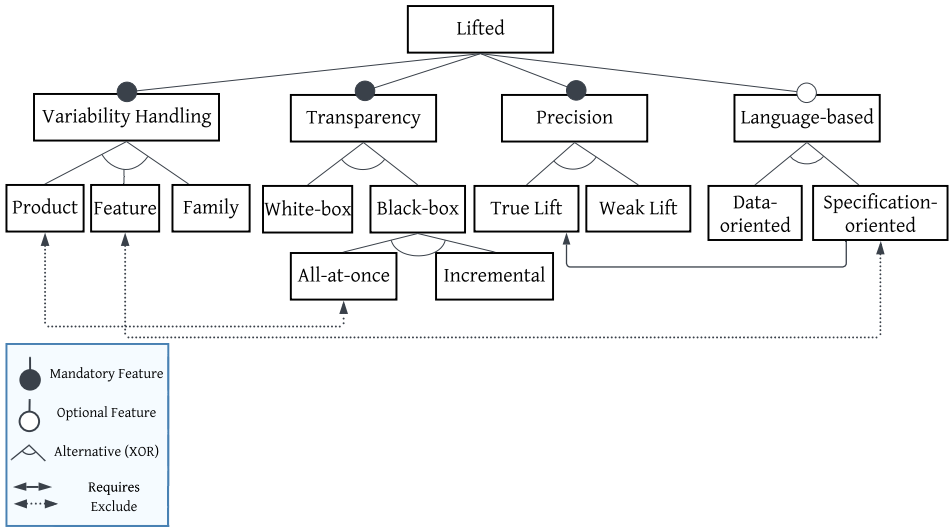

Fig. 6. A refined feature model for lifted analyses, including cross-cutting constraints identified from our SLR.

1. Product-based lifts and all-at-once wrappers are mutually exclusive. That is, we found no occurrences of product-based analyses using an all-at-once wrapping strategy for black-box lifting. This makes sense, as (by definition) a black-box product-based analysis must combine multiple analysis results from separate products. It follows that, if a black-box strategy is to be used, it must be incremental.
2. Feature-based and specification-oriented lifts are mutually exclusive. That is, we found no occurrences of a feature-based analysis which relied primarily on specification rewriting. This may be explained by the fact that feature-based lifts, like product-based lifts, attempt to generalize from multiple analysis results (i.e., of the feature modules) to obtain an SPL-level result. By contrast, specification rewrites (as defined above) result in a single variability-aware specification which lifts the analysis for the entire product line. One could potentially imagine an alternative form of specification rewriting, in which multiple analyses are generated for each feature module, but we have not identified any lifts of this form in our survey.
3. Specification-oriented lifting always requires a true lift. That is, every specification-oriented lift identified in our survey is a true lift, rather than a weak lift. Once again, there is no reason *in principle* why a specification rewrite cannot generate a weak lift. Instead, this may be explained by the fact that, due to the technical complexity of defining a rewriting process for lifting, such lifts are only pursued with the intention of defining a true lift.

Fig. 6 illustrates a refined version of our feature model with these cross-cutting constraints made explicit. In total, this new feature model allows for 39 valid configurations, compared to the original 54. The configurations identified in our survey therefore cover 62% of the configuration space of the refined feature model, compared with 44% for the initial feature model. We emphasize that by including these constraints, we do not assert that these combinations of features could not be employed for lifting *in principle*, only that they have not been identified in our survey.

5.3. RQ2: Overall frequencies of lifting features

Having identified which subset of valid configurations were actually implemented in practice, we next calculate the relative frequencies of each of the features and feature configurations. Fig. 7 shows the relative frequencies of each feature in the feature model compared with its alternatives. We can make several observations. First, we observe

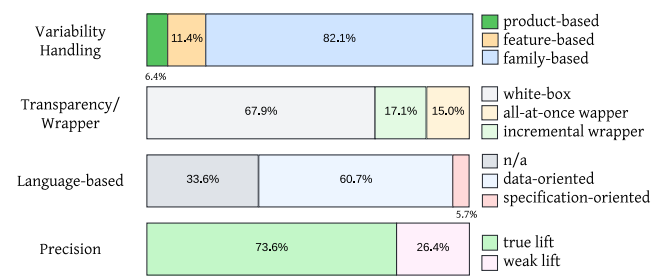

Fig. 7. Relative frequencies of each of the features across all papers identified in our survey.

that family-based analyses form a super-majority (82.1%) of all lifted analyses. This likely reflects the same observations made by Thüm et al. (2014) that the relative complexity of implementing family-based lifts is warranted by their relative improvements in performance compared to most feature-based and product-based analyses. With respect to the two other forms of lifting, we see a slightly more frequent occurrence of feature-based lifts compared to product-based lifts.

With respect to the transparency of lifts, we still see a super-majority (67.9%) of lifts being white-box, although this preference is less dramatic than the one for family-based analyses. This preference likely reflects the general notion proposed by Kästner et al. (2012b) that keeping variability local throughout the analysis (in a white-box lift) can provide better opportunities for intelligently leveraging commonalities across the product space to reduce analysis work. The stronger preference of black-box lifting, compared to feature- and product-based, may reflect that (i) from an engineering perspective, it is very valuable to reuse existing analyses in a black-box fashion, and (ii) through techniques such as variability encoding, it is possible to still perform family-based analyses in a black box fashion.

With respect to the relevance of specification languages in analysis lifting, we see that one third of analyses are not considered language-based — that is, their only input is an SPL, without any additional criteria or specification of the analysis. Of the remaining analyses, the overwhelming majority are data-oriented, as opposed to specification-oriented, lifts. This likely reflects the fact that most commonly used specification languages – such as temporal logic properties used in model-checking – are not sufficiently expressive to make lifting by specification-level rewriting feasible.

Table 4
Frequencies of all configurations for lifted analyses which appear more than once in our survey (n = 140). Feature identifiers: FM = Family-based, FT = Feature-based, P = Product-based, WB = White-box, AW = All-at-once wrapper, IW = Incremental wrapper, DO = Data-oriented, SO = Specification-oriented, TL = True Lift, WL = Weak lift.

Configuration	Frequency	Configuration (cont'd)	Frequency
$\{FM, WB, DO, TL\}$	24.29%	$\{FT, IW, WL\}$	2.86%
$\{FM, WB, TL\}$	20.00%	$\{FT, WB, WL\}$	2.14%
$\{FM, WB, DO, WL\}$	7.14%	$\{FT, WB, DO, TL\}$	2.14%
$\{FM, AW, DO, TL\}$	6.43%	$\{P, WB, DO, TL\}$	2.14%
$\{FM, AW, DO, WL\}$	6.43%	$\{FM, AW, SO, TL\}$	1.43%
$\{FM, IW, DO, TL\}$	4.29%	$\{FM, IW, DO, WL\}$	1.43%
$\{FM, IW, TL\}$	3.57%	$\{FT, WB, DO, WL\}$	1.43%
$\{FM, WB, WL\}$	2.86%	$\{FT, IW, TL\}$	1.43%
$\{FM, WB, SO, TL\}$	3.57%	$\{P, IW, TL\}$	1.43%

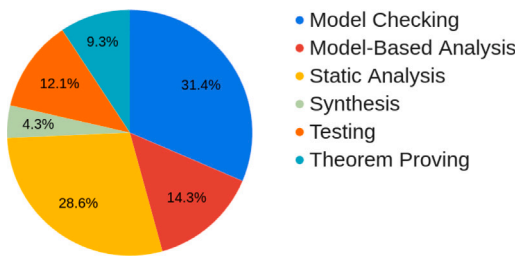

Fig. 8. Overall breakdown of analyses classes.

Finally, we see that nearly three-quarters (73.6%) of the lifts are classified as *true lifts*, meaning that they are meant to meet the correctness specification of Definition 2.2. The remaining 26.4% are weak lifts, meaning that they are sound but do not provide complete results, equivalent to those produced by brute-forced analysis (cf. Definition 3.1).

Having identified the relative frequency between each of the alternatives in each class of features, we can next move to identifying the relative frequency of feature configurations. Table 4 shows the 18 most frequently represented configurations in our set of surveyed papers. Each of the remaining six configurations had only one representative in the dataset, and so they are omitted from the table. We see that family-based white-box lifts collectively form over half (51.4%) of lifted analyses, with just under half (44.29%) also being true lifts. Family-based all-at-once black-box lifts also form a relatively popular combination, collectively accounting for 14.29% of lifts. The remainder of the surveyed papers are distributed relatively evenly across the rest of the configuration space.

In summary, while there are certain features and combinations of features which are disproportionately favoured in lifting – such as white-box lifting and family-based analysis – there is also a significant diversity in the methods which have been used in the research literature.

5.4. RQ3: Variations across analysis classes

In our investigation of **RQ2**, we analysed the relative frequency of features and feature configurations across the entire set of papers collected in our survey. We now turn to considering each of the six analysis classes identified in Section 4.4 to see whether the distribution of lifting techniques varies from these general trends.

We begin by reporting on the overall frequency of each analysis class. Fig. 8 shows the relative size of each class of analyses within our set of surveyed papers. The most commonly lifted analysis in our survey is model checking (31.4%), followed narrowly by static code analyses (28.6%). Model-based analyses and testing are relatively evenly represented (14.3% and 12.1%, respectively). Theorem proving accounts for 9.3% of the lifted analyses, with synthesis accounting for 4.3%.

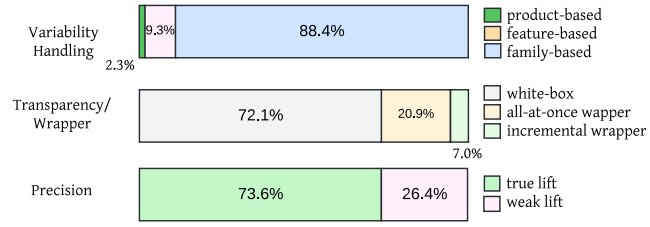

Fig. 9. Relative frequencies of features used for lifted model checking (n = 43).

We now consider each of these classes in turn, from the largest (model checking) to the smallest (synthesis). For each class, we perform the same analysis in **RQ2**: we first measure the relative frequency between each set of lifting alternatives, and then consider the most popular feature configurations for each class. Where appropriate, we provide some historical information and examples from the research literature to contextualize our observations.

Model checking. Fig. 9 shows the relative proportions of features in lifted model checkers. We do not show the proportions for the language-based features, as (i) model-checking is always a language-based analysis, but (ii) specification languages for model-checking (such as LTL) are insufficiently expressive to perform lifting through specification rewriting. As such, all lifted model checking is a case of data-oriented lifting. As with analyses in general, family-based and white-box lifts are the most commonly employed for model checking, with the proportions being slightly larger than in the set of all analyses. An early breakthrough in white-box lifted model checking was developed by Classen et al. (2010). The authors introduced Featured Transition Systems as a variability-aware analog of traditional Labelled Transition Systems, and redefined a traditional automata-theoretic algorithm for verifying regular and ω-regular properties, the key difference being the production of a *set* of counterexamples for any violating product, rather than halting after a single counterexample.

After white-box lifting, all-at-once wrappers account for the second most common form of lifting for model checkers by a considerable margin; this is not the case for analyses in general, for which incremental wrappers are slightly more common (cf. Fig. 7). The earliest instance of a all-at-once approach to model-checking was given by Post and Sinz (2008b) in 2008. This work introduced the variability embedding for annotated imperative programs which has since become commonplace in all-at-once-based wrapping. The embedding was used at the time for bounded model checking of safety properties, but the authors noted that the construction could be reused for other forms of analysis. A all-at-once approach to CTL model checking of featured transition systems (FTS) was implemented by Classen et al. (2011, 2014) on top of the NuSMV symbolic model checker. In their implementation, feature variables are embedded in the model as Boolean state variables, and feature-annotated transitions are encoded into the symbolic transition relation. Only a small modification of the NuSMV model checking algorithm is made, in order to distinguish feature variables from ordinary state variables to allow counterexamples to be mapped to specific configurations of features. The set of initial states is also generalized to allow for any valid configuration of features. Similar encodings of variability-aware models have been used for the model checker mCRL2 (ter Beek et al., 2017). Compared to the set of all analyses, we see that weak lifted model checkers relatively commonplace. This may reflect a general preference towards providing sound verification, rather than complete sets of counterexamples, as well as a desire to reuse existing product-based model checkers.

Static analyses. Fig. 10 shows the relative frequency of the features for lifted static analyses. The distributions for variability handling and lift transparency are very similar to those found in model checking, with family-based (87.5%) and white-box (70.0%) lifts being very

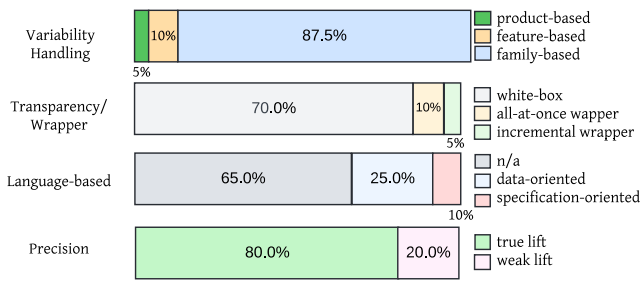

Fig. 10. Relative frequencies of features used for lifted static analyses ($n = 40$).

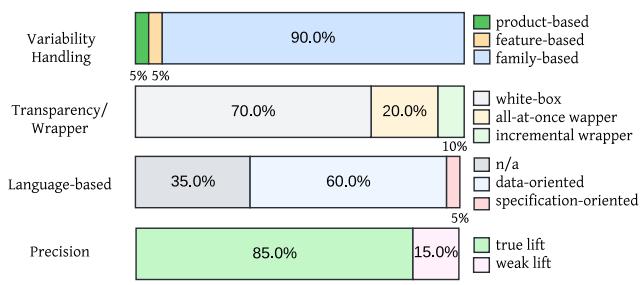

Fig. 11. Relative frequencies of features used for lifted model-based analyses ($n = 20$).

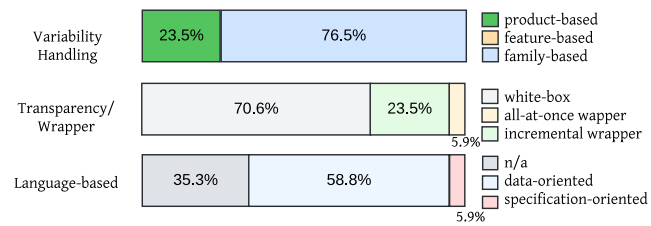

Fig. 12. Relative frequencies of features used for lifting testing ($n = 17$).

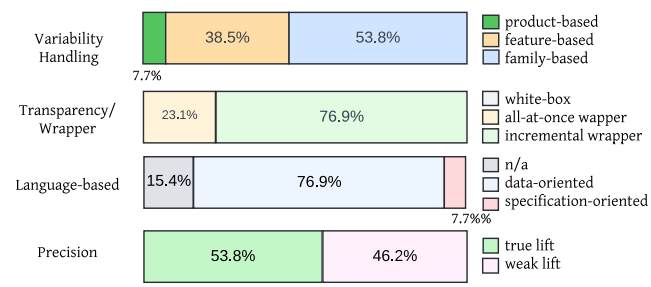

Fig. 13. Relative frequencies of features used for lifted theorem proving ($n = 13$).

popular. In our survey, one of the most extensively studied subclasses of lifted static analyses is type-checking. Examples of white-box lifted type systems exist for imperative (Kästner et al., 2012b; Schaefer et al., 2011) and functional programming languages (Chen et al., 2014). In the annotative paradigm, Kästner et al. (2012a) provide a type system for an annotative extension of Featherweight Java (FJ). The lifted type system extends the typing context of FJ with a feature annotation, and uses this additional information to perform well-formedness checks on annotated program terms. Also in the annotative paradigm, Chen et al. (2014) define an extension of λ-calculus, VLC, which can be used as a pure functional programming language. In addition to a extended typing system using rules for checking type equivalences, the authors also provide a type inference procedure for VLC.

Another example of a white-box lifted static analysis is dead code detection, an important analysis used for compiler optimization. Angerer et al. (2014) provide a white-box lift of this analysis for annotative programs. The analysis relies on a variational system dependence graph computed from the annotated program's AST. A variability-aware reachability analysis is then performed to identify unreachable code fragments.

Model-based analyses. Fig. 11 shows the relative frequency of features for model-based analyses. Like model checking and static analyses, model-based analyses follow a similar pattern of emphasizing family-based and white-box methods. Examples of white-box lifted model-based analyses include performance analysis (Kowal et al., 2015), model consistency checking (Alférez et al., 2011), and change impact analysis (Shahin et al., 2021). Compared to static code analyses, model-based analyses employ black box methods more frequently. One of the earliest lifts identified in our SLR was a framework for lifted OCL constraint checking due to Czarnecki and Pietroszek (Czarnecki and Pietroszek, 2006).

Testing. Fig. 12 shows the relative proportions of features for lifted testing and testing-related analyses. Unlike the preceding classes of analyses, testing-related analyses are more likely to employ product-based reasoning. Examples of such cases include regression testing (Jung et al., 2020) and model-based integration testing (Kowal et al., 2015). We identified no cases of lifted feature-based testing, nor any weak lifts. A particularly important form of analysis for testing is dynamic execution, with white-box and black-box implementations of variational executions having been developed as discussed above in Section 3.2.

Theorem proving. Fig. 13 shows the relative proportions of features for theorem proving. Theorem proving is unique among the six analysis classes for its complete absence of white-box implementations — all instances of theorem proving identified in our survey employ black-box methods. This may be explained by the fact that modern theorem provers are generally very complex and highly optimized, making white-box lifting less attractive from an engineering perspective. We can imagine, for instance, that a satisfiability solver built from scratch using a variability-aware redefinition of the DPLL algorithm, but which lacked the heuristics and optimizations used in modern SAT solving, may not provide sufficient performance.

Early investigations into deductive verification for SPLs focused on either incremental verification of feature modules (Thüm et al., 2011), or a all-at-once approach. For instance, Thüm et al. developed a procedure for generating a all-at-once (Thüm et al., 2012) of JML-annotated SPLs, which could then be verified using the KeY theorem prover (Ahrendt and Grebing, 2016). Incremental wrappers for deductive theorem proving have since been studied for delta-based product lines, which generally seek to verify each delta in isolation and then compose the results (Damiani et al., 2012; Scaletta et al., 2021). More recently, incremental approaches for verifying compositional product lines based on *partial proofs* (Kodetzki et al., 2024) and *proof plans* (Kuiter et al., 2022) have been proposed. Partial proofs are used to decompose the verification problem for the SPL into subproofs for individual methods, which can then be recombined. Proof plans are a particular form of partial proof-based verification, in which a proof graph is used to generate all possible combinations of method-level subproofs for the SPL, such that deterministic proof plans (subgraphs) can be identified and leveraged fro proof reuse. Verification procedures built on partial proofs can be viewed as incremental wrappers, since the underlying verification is procedure is the same for the subproofs. Another significant advantage of these incremental approaches to proof engineering is that the resulting proofs are much more amenable to reuse over the evolution of the SPL (Kuiter et al., 2022).

Wang et al. (2023) provide a incremental wrapper for lifting the functional equivalence checker CLEVER (Mora et al., 2018). The functional equivalence problem takes two programs P_1, P_2 over the same

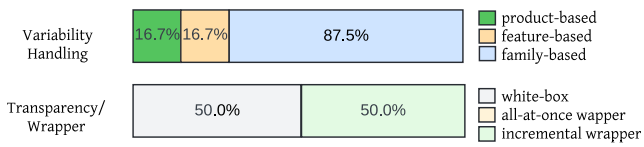

Fig. 14. Relative frequencies of features used for lifted synthesis ($n = 6$).

Table 5
All configurations used for lifted synthesis ($n = 6$). Note that each of the bottom three rows represent a single analysis.

Configuration	Frequency
$\{FM, WB, TL\}$	50.0%
$\{FM, IW, TL\}$	16.7%
$\{FT, IW, TL\}$	16.7%
$\{P, IW, TL\}$	16.7%

Table 6
Comparing the frequency with which lifts are accompanied by correctness proofs for different lifting strategies (i.e., configurations).

Configuration	# Instances	Proof	No Proof
$\{FM, WB, DO, TL\}$	34	44.1%	55.9%
$\{FM, WB, TL\}$	28	39.3%	60.7%
$\{FM, WB, DO, WL\}$	10	50.0%	50.0%
$\{FM, AW, DO, WL\}$	9	11.1%	88.9%
$\{FM, AW, DO, TL\}$	9	66.7%	33.3%
$\{FM, IW, DO, TL\}$	6	83.3%	16.7%
$\{FM, IW, TL\}$	5	20.0%	80.0%

input variables $\bar{x}$ and output variables $\bar{y}$, and attempts to find some assignment to inputs such that the two programs will produce distinct outputs. To lift the equivalence checker for annotative programs, the wrapper generates a metaproduct using the standard variability embedding procedure defined by Kästner at al (Kästner et al., 2012b). The non-lifted equivalence checker is then used to produce a counterexample to the equivalence, for which a feature annotation is extracted, representing the set of products for which the counterexample applies. The metaproduct is then pruned and the process is repeated until equivalences have been either verified or disproven with counterexamples.

Synthesis. We conclude with an overview of techniques for lifting synthesis to product lines. Of the six analysis classes, synthesis is the one which has been lifted the least, with only 6 instances identified in our SLR. The relative frequencies of the features is shown in Fig. 14. Approaches to synthesis have so far remained balanced between white-box and black-box methods. An example of the former is supervisory control synthesis (Basile, 2019), while an example of the latter is incremental game-based synthesis of Buchi automata (Greenyer et al., 2013). We also refer the reader back to Section 3.2 for a discussion of comparisons between black-box and white-box synthesis techniques. Since there are so few instances, we provide a table containing all configurations used for lifting synthesis identified in our SLR (Table 5).

5.4.1. RQ4: Lifting strategies and correctness proofs

To gauge the degree to which lifted analyses have been validated for correctness, we investigated each of the surveyed papers to see how often a formal proof accompanied a new lifted analysis, and compared the results for different lifting strategies. First, we emphasize that there are many reasons why a proof of correctness may not be included in a published work, such as the resource constraints and priorities of the authors. As such, our results here are not meant to conclusively show that certain lifting strategies are easier to verify than others; rather, we expect that *if such trends exist*, they could be identified through such an investigation.

To establish a baseline, we first calculated the rate at which correctness proofs were provided across all papers in our SLR. We found that 52% of the papers in our survey provided a correctness proof, while 48% did not. We then compared this (nearly balanced) baseline against specific configurations identified in our survey. However, such a comparison is only plausible if there are sufficiently many instances of each configuration. As such, we considered only specific configurations corresponding to five or more papers in our SLR.

The results of this analysis are shown in Table 6. The results are mixed. The most frequent combinations of features – white-box, family-based lifts – provide relatively balanced proportions between lifts with proofs and those without. The remaining configurations – all black-box lifts – skew significantly to one direction or the other. We note that all-at-once wrappers which are weak lifts are *less* likely to be accompanied by correctness proofs. This at first seems counter-intuitive, since proving a weak lift is theoretically simpler than proving a true lift. However, by the same token, because much of the correctness of an all-at-once wrapper is inherited directly by the correctness of the original analysis (which is assumed), the correctness of such lifts may be generally seen as *obvious*, thereby reducing the need to include such a proof.

5.5. Potential research directions

Our SLR also allows to identify some research gaps and thus suggest future research directions. Based our SLR results, we can see two notable gaps: a lack of white-box methods in theorem proving, and a general lack of techniques for SPL synthesis. We now propose some ideas for potential future work in addressing these gaps.

White-box proof production. One gap we see in the research literature is a lack of white-box lifting of theorem proving procedures. As mentioned, this is understandable given the remarkable complexity and optimizations in modern theorem-proving tools. Nevertheless, we can consider some potential opportunities for white-box approaches. One important task in theorem proving is the construction of proofs for unsatisfiable formulae. At the time of writing, the only variability-aware theorem prover which computes unsatisfiability proofs is the one by Young et al. (2020), which is an incremental wrapper of traditional solvers' proof-producing procedures. Because there is no sharing within the proofs themselves, this may lead to unnecessarily large variational unsatisfiability proofs. Recent advances in proof formats, such as proof skeletons (Reeves et al., 2023) may provide inspiration for more efficient white-box representations of variational proofs.

A rewriting system for dependently-typed proofs. Dependently-typed proof assistants such as Coq (Paulin-Mohring, 2011) and Lean (Moura and Ullrich, 2021) both implement the propositions-as-types theorem proving paradigm. In this paradigm, proofs are identified with programs, e.g., a proof of proposition a P is a term $t : P$, and proofs about objects can be interpreted as functions. For instance, consider a proof π asserting that, for any sets s and t, we have $s \cap t = t \cap s$. This proof is implemented as a function which takes sets s and t as input, and then returns a witness of type $s \cap t = t \cap s$ as output. If one then defines a type of variational set (Shahin et al., 2021) with an intersection operator, we should be able to mechanically rewrite the proof π into an equivalent proof for lifted sets. This assumes, of course, that the lifted intersection is correctly defined. It may also be possible to take inspiration from previous work on lifting functional programs (Shahin and Chechik, 2020) and on proof repair (Ringer et al., 2021) to lift more complex proofs to the product line level.

Incremental wrappers for AI-driven SPL synthesis. Overall, synthesis is one of the problems which is most under-represented in terms of lifting. One exciting opportunity for new work is driven by the recent emergence of generative AI models such as GPT-4. Modern generative AI models have been shown to be performant at program synthesis (e.g., in GitHub Copilot) compared to traditional synthesis methods.

A natural question is to ask whether such tools can form the basis of lifted synthesis for SPLs. Indeed, preliminary investigations by Acher et al. (2023) showed that GPT-4 is able to produce reasonable, simple annotated programs from natural language prompts. Based on this, we can treat a generative AI model as a black box synthesis algorithm and sketch an incremental wrapper-based approach to AI-driven SPL synthesis following the ideas of traditional counterexample-guided inductive synthesis (CEGIS). We can consider providing a (variational) synthesis specification to the AI model, which will then attempt to synthesize an SPL. Using a variability-aware verifier, we can aim to determine for which (if any) configurations the synthesis problem has been satisfied. If there are any successful configurations, a snapshot of the SPL is captured, and further synthesis continues with a refined input to the model, describing the configurations which are yet to be covered.

5.5.1. Summary

In this section, we have used the data collected from our SLR to investigate 5 research questions based on our proposed classification scheme for lifted analyses. Research questions **R1** and **R2** were intended to help us complement the theoretical derivation of our feature model for lifting (Fig. 3) with empirical data from real-world lifted analyses. In doing so, we observed that our initial feature model significantly overestimated the number of lifting strategies which were pursued in practice, meaning that there are further . We identified some of these (simpler) constraints based on the SLR data, and included them in a refined feature model, but further study is required to develop a more fine-grained understanding of which other constraints may be relevant to include. Research questions **RQ3–RQ5** leveraged our classification scheme to identify patterns in the research literature, such as the popularity of specific feature combinations for particular kinds of analyses, as well as gaps which may represent opportunities for new research.

6. Discussion

The development of our classification scheme and its evaluation via an SLR have highlighted several important points. First, we have strong evidence to support the usefulness of viewing lifting as a *structured process*, i.e., one in which developers can make various decisions about how to implement their lift. By imposing this structured perspective on the wide landscape of lifted analyses through our SLR, developers can approach the design phase of lifting by (i) determining which design choices (*features*) they want to prioritize, and (ii) identifying analyses which have combined these particular features to use as exemplars. Moreover, our survey has revealed that there is not just one ideal lifting method which is best suited for all purposes; different lifting methods have been used in each of the analysis domains we studied, and the tradeoffs predicted by our taxonomy can be demonstrated by case studies from the literature.

6.1. Threats to validity

Like any SLR, our investigation and its conclusions are threatened by the potential incompleteness of our data, and from our manual evaluation process. With respect to the first point, our combination of database searching and bidirectional snowballing follows the recommended guidelines for SLRs (Wohlin, 2014; Wohlin et al., 2022). Moreover, we do not claim that our survey set represents every lifted analysis, only that it is a sufficient sample size to conduct a meaningful analysis. As a form of *post hoc* cross-validation, we investigated a list of papers against a publicly available repository of SPL analyses maintained by the authors of a 2014 survey (Thüm et al., 2014). This repository consists of over 300 works collected from that survey, as well as many papers published after its publication. The scope of the repository is broader than that of this article, as it includes all forms of SPL analysis, not only lifted analyses. To the best of our knowledge, the papers added to this repository since the publication of the original 2014 article were not selected as part of an SLR, so we cannot comment on the completeness of the repository after 2014. But in any case, it offers a useful benchmark with which to compare our data. When we investigated this repository, we did not find any cases of works which met our criteria for inclusion, and which were not duplicated by some paper in our surveyed set. Thus we have a strong reason to believe our surveyed papers represent a sufficiently comprehensive view of the landscape of lifted analyses to make our observations.

With respect to manual evaluation, it is possible that we misinterpreted or misunderstood some aspects of the implementation of lifts when reading the selected papers. However, this threat to validity really applies only to **RQ1**, i.e., the completeness of our taxonomy. As discussed in Section 5.3, our classification was made based on relatively well-defined and objective criteria, and we did not find many cases of ambiguities in applying our classification scheme. As such, we have reasonable confidence that most of the classifications we made are correct.

6.2. Future work

While we believe that our taxonomy is complete, it may not be sufficiently refined. Within each of the classes of lifting methods in our taxonomy, there may be further implementation patterns that give a finer-grained view of the lifting process. Another potential avenue for future work is facilitating the development of formal correctness proofs for lifted analyses. We believe there are several ways in which this could be studied. A first step would be to do this for *compositional* lifted analyses, i.e., lifted analyses which are composed of other lifted analyses. One easy consequence of Definition 2.2 is that composing lifted functions preserves compositionality. In previous work, we proved that lifting preserves a general class of functional correctness properties (Murphy et al., 2023), so a general procedure for rewriting a certain class of correctness proofs should be within reach. To prove more general lifted analyses, we may also be able to apply our taxonomy to define *proof patterns* for the various lifting methods. For instance, we may find that correctness proofs for metaproduct or modular wrappers also follow predictable patterns. If this is the case, we may be able to implement customized proof automation and tactics to support the specification and verification of lifted analyses in proof assistants.

7. Related work

We conclude by summarizing two areas of related work: (1) alternative classification schemes for SPL analyses, and (2) other SLRs conducted in the SPLE domain.

7.1. Classifying SPL analyses

All SPL-level analyses, including lifted analyses, can be distinguished from one another according to how they exploit commonalities amongst products to reduce the analysis work. This is the basis for the classification scheme for variational analyses developed by Thüm et al. (2011, 2014), and it distinguishes between product-based, family-based, and feature-based analyses. Our classification scheme refines the one proposed by Thüm et al. adding additional orthogonal dimensions of variability between lifted analyses, with the intention of providing a more comprehensive perspective on choices made when implementing a lift.

Castro et al. proposed a formal framework for studying the compositional nature of SPL analyses. They argue that important properties of the analyses (e.g., compositionality properties, intermediate analysis steps) can only be understood by considering particular kinds of tasks. The authors present a framework for studying properties of variability-aware analyses of Discrete-Time Markov Chains. Within it, they are able

to reason about the soundness and compositionality of analyses. Here, we take a different view, motivated by the assumption that general characteristics of lifting as a *process* can be defined and observed (e.g., in terms of features of lifted analyses).

7.2. Systematic literature reviews for SPLE

Our SLR is one of several reviews which have been conducted in the product line domain in recent years. Like ours, many of these SLRs propose conceptual models to help organize otherwise varied and disparate topics in SPLE. In 2010, Benavides et al. (2010) conducted a review of feature model analyses, using a corpus of 53 papers. Like us, they proposed a high-level conceptual framework for comparing feature model analyses. As mentioned above, Thüm et al. (2014) conducted a systematic review of 123 product line analyses, classifying them as either product-based, feature-based or family-based. A 2012 survey on testing methods for SPLs examined 15 studies, classified into eight testing methodologies (Lee et al., 2012). In 2016, Kumar et al. (2016) conducted an SLR of test case prioritization methods for SPLs, and classified 90 papers into 8 forms of test prioritization. Given the breadth of our scope, our SLR is on the larger end of the spectrum relative to other SLRs in SPLE.

8. Conclusion

In this article, we presented a classification scheme for software analyses which have been lifted from products to product lines. Our scheme is based on several *features* of lifted analyses, such as the chosen level of abstraction for handling software variability, the degree to which the original analysis is reused as a subprocedure, and the precision of lift. To evaluate and apply our classification scheme against the wide landscape of existing lifted analyses, we conducted a systematic review of the literature on lifted product line analyses, resulting in a corpus of 140 papers. We were able to identify patterns in these lifted analyses, such as the absence of certain combinations of features, and the ways in which particular features of lifting have been favoured when implementing certain classes of analyses. It is our hope that our taxonomy and the accompanying SLR will serve as a useful reference for extant lifted analyses, and will provide engineers and researchers with guidance on how to implement and validate the lifting of their own analyses.

CRediT authorship contribution statement

Logan Murphy: Writing – review & editing, Writing – original draft, Visualization, Methodology, Investigation, Data curation, Conceptualization. **Mahmood Saifi:** Writing – original draft, Data curation, Conceptualization. **Alessio Di Sandro:** Writing – review & editing, Writing – original draft, Conceptualization. **Marsha Chechik:** Writing – review & editing, Writing – original draft, Supervision, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors are supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) and by General Motors, Canada.

Data availability

A link to the data pertinent to the paper is included.

References

Acher, M., Duarte, J.G., Jézéquel, J.-M., 2023. On programming variability with large language model-based assistant. In: Proceedings of the 27th ACM International Systems and Software Product Line Conference-Volume A. pp. 8–14.

Ahrendt, W., Grebing, S., 2016. Using the KeY prover. In: Deductive Software Verification–the KeY Book: From Theory To Practice. Springer, pp. 495–539.

Alférez, M., Lopez-Herrejon, R.E., Moreira, A., Amaral, V., Egyed, A., 2011. Supporting consistency checking between features and software product line use scenarios. In: Top Productivity Through Software Reuse: 12th International Conference on Software Reuse, ICSR 2011, Pohang, South Korea, June 13-17, 2011. Proceedings 12. Springer, pp. 20–35.

Angerer, F., Prähofer, H., Lettner, D., Grimmer, A., Grünbacher, P., 2014. Identifying inactive code in product lines with configuration-aware system dependence graphs. In: Proceedings of the 18th International Software Product Line Conference-Volume 1. pp. 52–61.

Badami, M., Benatallah, B., Baez, M., 2023. Adaptive search query generation and refinement in systematic literature review. Inf. Syst. 102231.

Baier, C., Katoen, J.-P., 2008. Principles of Model Checking. MIT Press.

Basile, D., 2019. Applying supervisory control synthesis to priced featured automata and energy problems. Int. J. Softw. Tools Technol. Transf. 21, 679–689.

Beek, M.H.t., Damiani, F., Lienhardt, M., Mazzanti, F., Paolini, L., 2019. Static analysis of featured transition systems. In: Proceedings of the 23rd International Systems and Software Product Line Conference-Volume a. pp. 39–51.

Benavides, D., Segura, S., Ruiz-Cortés, A., 2010. Automated analysis of feature models 20 years later: A literature review. Inf. Syst. 35 (6), 615–636.

Bodden, E., Tolêdo, T., Ribeiro, M., Brabrand, C., Borba, P., Mezini, M., 2013. Spllift: Statically analyzing software product lines in minutes instead of years. ACM SIGPLAN Notices 48 (6), 355–364.

Brabrand, C., Ribeiro, M., Tolêdo, T., Borba, P., 2012. Intraprocedural dataflow analysis for software product lines. In: Proceedings of the 11th Annual International Conference on Aspect-Oriented Software Development. pp. 13–24.

Bürdek, J., Lochau, M., Bauregger, S., Holzer, A., Von Rhein, A., Apel, S., Beyer, D., 2015. Facilitating reuse in multi-goal test-suite generation for software product lines. In: Fundamental Approaches To Software Engineering: 18th International Conference, FASE 2015, Held As Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015, Proceedings 18. Springer, pp. 84–99.

Castro, T., Teixeira, L., Alves, V., Apel, S., Cordy, M., Gheyi, R., 2021. A formal framework of software product line analyses. ACM Trans. Softw. Eng. Methodol. (TOSEM) 30 (3), 1–37.

Chen, S., Erwig, M., Walkingshaw, E., 2014. Extending type inference to variational programs. ACM Trans. Program. Lang. Syst. 36 (1), http://dx.doi.org/10.1145/2518190.

Classen, A., Cordy, M., Heymans, P., Legay, A., Schobbens, P.-Y., 2012. Model checking software product lines with SNIP. Int. J. Softw. Tools Technol. Transf. 14, 589–612.

Classen, A., Cordy, M., Heymans, P., Legay, A., Schobbens, P.-Y., 2014. Formal semantics, modular specification, and symbolic verification of product-line behaviour. Sci. Comput. Program. 80, 416–439.

Classen, A., Cordy, M., Schobbens, P.-Y., Heymans, P., Legay, A., Raskin, J.-F., 2013. Featured transition systems: Foundations for verifying variability-intensive systems and their application to LTL model checking. IEEE Trans. Softw. Eng. 39 (8), 1069–1089. http://dx.doi.org/10.1109/TSE.2012.86.

Classen, A., Heymans, P., Schobbens, P.-Y., Legay, A., 2011. Symbolic model checking of software product lines. In: Proceedings of the 33rd International Conference on Software Engineering. ICSE '11, Association for Computing Machinery, New York, NY, USA, pp. 321–330. http://dx.doi.org/10.1145/1985793.1985838.

Classen, A., Heymans, P., Schobbens, P.-Y., Legay, A., Raskin, J.-F., 2010. Model checking lots of systems: efficient verification of temporal properties in software product lines. In: Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1. pp. 335–344.

Cordy, M., Davril, J.-M., Greenyer, J., Gressi, E., Heymans, P., 2015. All-at-once-synthesis of controllers from scenario-based product line specifications. In: Proceedings of the 19th International Conference on Software Product Line. pp. 26–35.

Cordy, M., Schobbens, P.-Y., Heymans, P., Legay, A., 2012. Behavioural modelling and verification of real-time software product lines. In: Proceedings of the 16th International Software Product Line Conference-Volume 1. pp. 66–75.

Czarnecki, K., Pietroszek, K., 2006. Verifying feature-based model templates against well-formedness ocl constraints. In: Proceedings of the 5th International Conference on Generative Programming and Component Engineering. pp. 211–220.

Damiani, F., Owe, O., Dovland, J., Schaefer, I., Johnsen, E.B., Yu, I.C., 2012. A transformational proof system for delta-oriented programming. In: Proceedings of the 16th International Software Product Line Conference-Volume 2. pp. 53–60.

Di Sandro, A., Shahin, R., Chechik, M., 2023. Adding product-line capabilities to your favourite modeling language. In: Proceedings of the 17th International Working Conference on Variability Modelling of Software-Intensive Systems (VaMoS'23). pp. 3–12.

Greenyer, J., Brenner, C., Cordy, M., Heymans, P., Gressi, E., 2013. Incrementally synthesizing controllers from scenario-based product line specifications. In: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering. pp. 433–443.

Greenyer, J., Kindler, E., 2013. Compositional synthesis of controllers from scenario-based assume-guarantee specifications. In: Model-Driven Engineering Languages and Systems: 16th International Conference, MODELS 2013, Miami, FL, USA, September 29–October 4, 2013. Proceedings 16. Springer, pp. 774–789.

Jung, P., Kang, S., Lee, J., 2020. Efficient regression testing of software product lines by reducing redundant test executions. Appl. Sci. 10 (23), 8686.

Kästner, C., Apel, S., Thüm, T., Saake, G., 2012a. Type checking annotation-based product lines. ACM Trans. Softw. Eng. Methodol. (TOSEM) 21 (3), 1–39.

Kästner, C., Von Rhein, A., Erdweg, S., Pusch, J., Apel, S., Rendel, T., Ostermann, K., 2012b. Toward variability-aware testing. In: Proceedings of the 4th International Workshop on Feature-Oriented Software Development. pp. 1–8.

Katoen, J.-P., 2016. The probabilistic model checking landscape. In: Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science. pp. 31–45.

Kiczales, G., Lamping, J., Mendhekar, A., Maeda, C., Lopes, C., Loingtier, J.-M., Irwin, J., 1997. Aspect-oriented programming. In: ECOOP'97—Object-Oriented Programming: 11th European Conference Jyväskylä, Finland, June 9–13, 1997 Proceedings 11. Springer, pp. 220–242.

Kodetzki, M., Bordis, T., Runge, T., Schaefer, I., 2024. Partial proofs to optimize deductive verification of feature-oriented software product lines. In: Proceedings of the 18th International Working Conference on Variability Modelling of Software-Intensive Systems. pp. 17–26.

Kowal, M., Tschaikowski, M., Tribastone, M., Schaefer, I., 2015. Scaling size and parameter spaces in variability-aware software performance models (t). In: 2015 30th IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 407–417.

Kuiter, E., Knüppel, A., Bordis, T., Runge, T., Schaefer, I., 2022. Verification strategies for feature-oriented software product lines. In: Proceedings of the 16th International Working Conference on Variability Modelling of Software-Intensive Systems. pp. 1–9.

Kumar, S., et al., 2016. Test case prioritization techniques for software product line: A survey. In: 2016 International Conference on Computing, Communication and Automation. ICCCA, IEEE, pp. 884–889.

Lee, J., Kang, S., Lee, D., 2012. A survey on software product line testing. In: Proceedings of the 16th International Software Product Line Conference-Volume 1. pp. 31–40.

Lity, S., Nieke, M., Thüm, T., Schaefer, I., 2019. Retest test selection for product-line regression testing of variants and versions of variants. J. Syst. Softw. 147, 46–63.

Manna, Z., Waldinger, R., 1980. A deductive approach to program synthesis. ACM Trans. Program. Lang. Syst. (TOPLAS) 2 (1), 90–121.

Meinicke, J., 2014. Varexj: a variability-aware interpreter for java applications. (Master's thesis). University of Magdeburg.

Meinicke, J., Thüm, T., Schröter, R., Benduhn, F., Saake, G., 2014. An overview on analysis tools for software product lines. In: Proceedings of the 18th International Software Product Line Conference: Companion Volume for Workshops, Demonstrations and Tools-Volume 2. pp. 94–101.

Mora, F., Li, Y., Rubin, J., Chechik, M., 2018. Client-specific equivalence checking. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering. ASE '18, Association for Computing Machinery, New York, NY, USA, pp. 441–451. http://dx.doi.org/10.1145/3238147.3238178.

Moura, L.d., Ullrich, S., 2021. The lean 4 theorem prover and programming language. In: Automated Deduction–CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings 28. Springer, pp. 625–635.

Murphy, L., Di Sandro, A., Shahin, R., Chechik, M., 2023. Reusing your favourite analysis framework to handle workflows of product line models. In: Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume a. SPLC '23, Association for Computing Machinery, New York, NY, USA, pp. 117–128. http://dx.doi.org/10.1145/3579027.3608983, URL https://doi-org.myaccess.library.utoronto.ca/10.1145/3579027.3608983.

Nguyen, H.V., Kästner, C., Nguyen, T.N., 2014. Exploring variability-aware execution for testing plugin-based web applications. In: Proceedings of the 36th International Conference on Software Engineering. pp. 907–918.

Paulin-Mohring, C., 2011. Introduction to the coq proof-assistant for practical software verification. In: LASER Summer School on Software Engineering. Springer, pp. 45–95.

Pierce, B.C., 2002. Types and Programming Languages. MIT Press.

Post, H., Sinz, C., 2008a. Configuration lifting: Verification meets software configuration. In: 2008 23rd IEEE/ACM International Conference on Automated Software Engineering. IEEE, pp. 347–350.

Post, H., Sinz, C., 2008b. Configuration lifting: Verification meets software configuration. In: 2008 23rd IEEE/ACM International Conference on Automated Software Engineering. pp. 347–350. http://dx.doi.org/10.1109/ASE.2008.45.

Prehofer, C., 1997. Feature-oriented programming: A fresh look at objects. In: European Conference on Object-Oriented Programming. Springer, pp. 419–443.

Reeves, J.E., Kiesl-Reiter, B., Heule, M.J., 2023. Propositional proof skeletons. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems. Springer, pp. 329–347.

Reps, T., Horwitz, S., Sagiv, M., 1995. Precise interprocedural dataflow analysis via graph reachability. In: Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. pp. 49–61.

Richters, M., Gogolla, M., 2000. Validating UML models and OCL constraints. In: International Conference on the Unified Modeling Language. Springer, pp. 265–277.

Ringer, T., Porter, R., Yazdani, N., Leo, J., Grossman, D., 2021. Proof repair across type equivalences. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. pp. 112–127.

Salay, R., Famelis, M., Rubin, J., Di Sandro, A., Chechik, M., 2014. Lifting model transformations to product lines. In: Proceedings of the 36th International Conference on Software Engineering. pp. 117–128.

Scaletta, M., Hähnle, R., Steinhöfel, D., Bubel, R., 2021. Delta-based verification of software product families. In: Proceedings of the 20th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences. pp. 69–82.

Schaefer, I., Bettini, L., Bono, V., Damiani, F., Tanzarella, N., 2010. Delta-oriented programming of software product lines. In: Software Product Lines: Going beyond: 14th International Conference, SPLC 2010, Jeju Island, South Korea, September 13-17, 2010. Proceedings 14. Springer, pp. 77–91.

Schaefer, I., Bettini, L., Damiani, F., 2011. Compositional type-checking for delta-oriented programming. In: Proceedings of the 10th International Conference on Aspect-Oriented Software Development, AOSD.11. pp. 43–56. http://dx.doi.org/10.1145/1960275.1960283.

Shahin, R., Chechik, M., 2020. Automatic and efficient variability-aware lifting of functional programs. Proc. ACM Program. Lang. 4 (OOPSLA), 1–27.

Shahin, R., Chechik, M., Salay, R., 2019. Lifting datalog-based analyses to software product lines. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 39–49.

Shahin, R., Kokaly, S., Chechik, M., 2021. Towards certified analysis of software product line safety cases. In: Computer Safety, Reliability, and Security: 40th International Conference, SAFECOMP 2021, York, UK, September 8–10, 2021, Proceedings 40. Springer, pp. 130–145.

Strüber, D., Born, K., Gill, K.D., Groner, R., Kehrer, T., Ohrndorf, M., Tichy, M., 2017. Henshin: A usability-focused framework for EMF model transformation development. In: Graph Transformation: 10th International Conference, ICGT 2017, Held As Part of STAF 2017, Marburg, Germany, July 18-19, 2017, Proceedings 10. Springer, pp. 196–208.

ter Beek, M.H., de Vink, E.P., Willemse, T.A.C., 2017. Family-based model checking with mCRL2. In: Huisman, M., Rubin, J. (Eds.), Fundamental Approaches To Software Engineering. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 387–405.

Thüm, T., Apel, S., Kästner, C., Schaefer, I., Saake, G., 2014. A classification and survey of analysis strategies for software product lines. ACM Comput. Surv. 47 (1), 1–45.

Thüm, T., Schaefer, I., Apel, S., Hentschel, M., 2012. Family-based deductive verification of software product lines. In: Proceedings of the 11th International Conference on Generative Programming and Component Engineering. pp. 11–20.

Thüm, T., Schaefer, I., Kuhlemann, M., Apel, S., 2011. Proof composition for deductive verification of software product lines. In: 2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops. IEEE, pp. 270–277.

Varró, D., Bergmann, G., Hegedüs, Á., Horváth, Á., Ráth, I., Ujhelyi, Z., 2016. Road to a reactive and incremental model transformation platform: Three generations of the VIATRA framework. Softw. Syst. Model. 15 (3), 609–629. http://dx.doi.org/10.1007/s10270-016-0530-4.

von Rhein, A., Thüm, T., Schaefer, I., Liebig, J., Apel, S., 2016. Variability encoding: From compile-time to load-time variability. J. Logical Algebr. Methods Program. 85 (1), 125–145.

Wang, A., Feng, N., Chechik, M., 2023. Code-level functional equivalence checking of annotative software product lines. In: Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume A. SPLC '23, Association for Computing Machinery, New York, NY, USA, pp. 64–75. http://dx.doi.org/10.1145/3579027.3608978, URL https://doi-org.myaccess.library.utoronto.ca/10.1145/3579027.3608978.

Weiser, M., 1984. Program slicing. IEEE Trans. Softw. Eng. (4), 352–357.

Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering. pp. 1–10.

Wohlin, C., Kalinowski, M., Felizardo, K.R., Mendes, E., 2022. Successful combination of database search and snowballing for identification of primary studies in systematic literature studies. Inf. Softw. Technol. 147, 106908.

Wong, C.-P., Meinicke, J., Lazarek, L., Kästner, C., 2018. Faster variational execution with transparent bytecode transformation. Proc. ACM Program. Lang. 2 (OOPSLA), 1–30.

Young, J.M., Walkingshaw, E., Thüm, T., 2020. Variational satisfiability solving. In: Proceedings of the 24th ACM Conference on Systems and Software Product Line: Volume a-Volume a. pp. 1–12.

Yovine, S., 1996. Model checking timed automata. In: School Organized By the European Educational Forum. Springer, pp. 114–152.

Logan Murphy is a Ph.D. student in Computer Science at the University of Toronto. His research interests include formal methods, in particular formal methods for software product lines and the application of theorem proving to software engineering.

Mahmood Saifi is an undergraduate student in Computer Science at the University of Toronto. His research interests include programming languages, formal methods and software product line engineering.

Alessio Di Sandro is a research engineer in the Software Engineering group at the University of Toronto. His interests include model management and software product line engineering.

Marsha Chechik is a Professor of Computer Science at the University of Toronto. Her research explores the use of formal methods to improve the quality of software. These include scalable automated verification technique; easy to read and review formal specification languages; verification of protocols; non-classical logics; and reasoning under inconsistency. She is also interested in model management and software product line engineering.